C64 Visual Assembler — User Manual

Version 2.3.8

A visual, block-based 6502 assembler for the Commodore 64. Build programs by dragging and dropping instruction blocks, and see the generated assembly and machine code in real time.

Table of Contents

- Map Editor (Multilayer Tilemaps)
- SID Editor (3-Voice Tracker)
- Curve Editor

Version 2.3.8 Highlights

- **Workspace save / open:** save the exact set of open file-backed tabs — including the active tab and each tab's editor mode — to a `.vaws` workspace file. Workspaces auto-save on change, and the app auto-restores your last workspace on launch.
- **Global memory panel toggle:** show or hide the full C64 memory panel from a dedicated UI switch.
- **Localized Ultimate Basic command reference:** command descriptions in the autocomplete popup and the Commands panel now follow the current UI language (Hungarian, English, Spanish, German, Dutch), with English fallback.
- **Refreshed Ultimate Basic graphics docs:** `COLOR PEN` and the plot/line/rect/circle and multicolor drawing command help text now match current compiler behavior.
- **Fixed KERNAL reference:** corrected the `SETLFS` and `PLOT` entries (addresses and calling conventions) in the disassembler's KERNAL address table.
- **Fixed memory usage with many tabs open:** per-tab undo/redo history is now capped (with a small debounce), preventing the unbounded memory growth that a long session with many open documents used to cause.
- **Editor toolbar cleanup:** removed the redundant breakpoint toggle buttons from the Expert and Ultimate Basic toolbars (breakpoints are still set from the line-number gutter), and aligned the Expert toolbar height with the Ultimate Basic toolbar.

---

1. Interface Overview

The app is split into three main panels:

Panel	Description
Left — Palette	All available instruction and macro blocks. Search or browse by category.
Center — Program	Your program. Drag blocks here, reorder them, edit operands.
Right — Output	Live ASM view and/or memory monitor output.

The mode badge at the far right of the header identifies the active **Block**, **Expert**, or **Ultimate Basic** editor. It updates immediately when the editing mode changes.

---

2. Block Palette

The palette on the left lists all available blocks grouped by category:

- **Data movement** — LDA, LDX, STA, STX, ...
- **Arithmetic** — ADC, SBC, INC, DEC, CMP, ...

- **Logic** — AND, ORA, EOR, BIT
- **Jumps & Branches** — JMP, JSR, RTS, BNE, BEQ, …
- **Register operations** — TAX, TAY, INX, DEX, …
- **Shift & Rotate** — ASL, LSR, ROL, ROR
- **Stack** — PHA, PHP, PLA, PLP
- **System** — CLC, SEC, NOP, BRK, …
- **Illegal instructions** — LAX, SAX, DCP, …
- **Structure** — LABEL, COMMENT, REGION, ENDREGION
- **Macros** — LOOP, NEXT, FOR, ENDF, PUSH, PULL, END, TEXT, BYTE, WORD, FILL, ALIGN, STRING, DATA, RAWBYTES, RAWTEXT, PETSCII, CHARSET, INCBIN, SID, INCLUDE, TABLE, ORG, MACRO, ENDM, INVOKE, IF, ELSE, ENDIF, VAR, WHILE, ENDW, REPEAT, UNTIL, MEMCPY, MEMSET, PRINT, PRINT_CHAR, PRINT_HEX, CLEAR_SCREEN, WAIT_KEY, DELAY, SET_BORDER, SET_BG, IRQ_SETUP, RAND, SPRITE_INIT, SPRITE_POS, WAIT_RASTER, JOYSTICK, MOUSE, SPRITE_COL, LOADFILE, REU_CHECK, REU_STASH, REU_FETCH, REU_SWAP, TURBO_SET, SUPERCPU_DETECT, TURBO_ENABLE, MAP_COPY, MAP_COPY16X16, SPRITE_ANIM, SCORE_BCD

Use the **search box** at the top of the palette to filter by name. Click the **Add selected block** button or drag a block into the program area.

---

3. Program Area

- **Drag & drop** blocks from the palette, or **reorder** existing blocks by dragging their handle (≡).
- Each block shows its **mnemonic**, **operand field**, and **addressing mode selector** (where applicable).
- Click the **▸ / ▾** toggle to collapse or expand a block.
- Use the **× (delete)** button on a block to remove it.
- **Collapse All** button folds all blocks at once.

Block panel minimap

The Program panel has a toggleable **minimap** button in its heading. When enabled, a narrow `56 px` canvas strip appears on the right edge of the panel, showing all blocks as colour-coded horizontal bars:

Bar colour	Block type
Cyan	Labels
Blue/purple	Macros and directives
Yellow	Instructions
Green	Comments and blank lines
Red	Blocks with a validation error

Collapsed blocks are rendered at reduced opacity. Click or drag anywhere on the minimap to scroll the program list to that position. The viewport indicator (accent-coloured rectangle) tracks the visible portion of the list. State is persisted in UI settings (`blockMinimap` key).

Operand input

- For branch/jump instructions (`BNE` , `JMP` , `JSR` , etc.) a **label picker** dropdown appears — click a defined label to insert it.
- Number format follows the **HEX / DEC** toggle in the toolbar (see section 5).

---

4. ASM View

The right panel shows the generated output in real time.

Output modes

Mode	Description
ASM	6502 assembly source with addresses and labels
Monitor	Hex / byte dump (C64 monitor style)
Disasm	Pure 6502 disassembly: address · hex bytes · mnemonics with resolved numeric operands. Macros are expanded to individual instructions (TEXT → LDA/STA pairs, LOOP → LDX, etc.). BYTE/WORD/FILL data shown as chunked hex dump. No macro names, comments, or annotations in output.
Both	ASM on top, monitor below
Disassembler	Same as Disasm — dedicated tab for the disassembly view
Toolkit	C64 reference panel: 16-colour palette swatch + PETSCII control-code and printable-character cheat sheet. Read-only — see the "Toolkit tab" subsection below for details.
Options	Program settings panel — number format, macro source toggle, debugger params

Toolkit tab

The **Toolkit** tab in the ASM view is a read-only quick-reference panel — it never modifies your program. Two sections:

Section	Content
C64 colour palette	16-swatch grid showing every C64 colour with its index (0–15 / `$00` – `$0F`) and name. Click a swatch to copy its hex index to the clipboard. Hover for the colour name (Light Blue, Brown, etc.).
PETSCII control codes	Common control codes for `CHROUT` ($FFD2): colour change codes (`$05` white, `$1C` red, `$1E` green, `$1F` blue, …), cursor movement (`$11` / `$1D` / `$91` / `$9D`), reverse on/off (`$12` / `$92`), `$93` clear screen, `$8E` / `$0E` charset switches. Also a printable-range cheat sheet (32-64 punctuation, 65-90 A-Z, 91-95 brackets, 96-127 graphics, 160-191 shifted graphics, 192-223 mirror).

The Toolkit is the fastest way to look up a colour index or a PETSCII control byte without leaving the editor.

Options tab

The **Options** tab contains the settings that affect code generation and output display:

- **Macro source** — when ON, macro definition blocks (MACRO…ENDM) show their source code inline in the ASM view.
- **Program start address** — now set via an **ORG block** in the program area rather than a separate input field. The first ORG block defines the program's load address; subsequent ORG blocks start additional sections at different

addresses.

- **Debugger params** — three inline toggles controlling which flags are passed to the external debugger on launch:
  - `-jmp` **ON/OFF** — jump directly to the program's start address after loading.
  - `-unpause` **ON/OFF** — unpause the debugger immediately on load.
  - `-wait` **ms ON/OFF** — adds a `-wait <ms>` delay before unpausing; select 500 ms or 1000 ms from the dropdown.
- **Compile info** — shows a summary of the compiled program (code start address, size, BASIC SYS stub status).

Clicking an ASM line

Click any line in the ASM view to **highlight the corresponding block** in the program area.

ASM line numbers

The ASM panel displays **line numbers** (`001 |`, `002 |`, …) to make troubleshooting easier when a compile error points to a specific line.

- The visual line numbers are for diagnostics only.
- **Copy ASM** still copies the clean source text **without** line-number prefixes.

Compile progress modal

During heavier actions, a centered progress modal appears with a progress bar:

- **Run in VICE** — compiling/building PRG and launching emulator.
- **Debug** — compiling/building PRG and launching debugger.
- **Load .asm file** — open an `.asm` file in Expert mode and materialize blocks from the source.

The modal closes automatically when the action completes or fails.

---

5. Settings & Toolbar

Control	Description
Number base (HEX / DEC / BIN)	Sets the display/input format for operands throughout the UI. BIN mode displays values as binary with `%` prefix (e.g. `%11111000`). The ASM view always shows each block in its own format.
Language	Switch the user interface between English, Hungarian, Spanish, German, and Dutch (Nederlands)
Theme	Light / Dark / OLED — select from the theme picker in the Settings menu. OLED uses a pure-black background for AMOLED displays.
CRT retro mode	Toggles a full-screen CRT filter: scanlines, phosphor vignette, flicker, and barrel distortion. State is saved between sessions.
Show memory panel	Global toggle that shows or hides the full C64 memory panel
BASIC SYS stub	Prepends a BASIC line that calls SYS to your program's origin
Sample	Load a built-in example program
Zoom in / out	Scale the block UI (affects all block elements)

Control	Description
Save Project	Save the current program as a `.json` project file
Save Program As	Save the current program as a `.json` project file using a new file dialog every time
Load Project	Load a previously saved project
Save Workspace	Save the exact set of currently open, file-backed tabs — including the active tab and each tab's editor mode (Block/Expert/Ultimate Basic) — to a `.vaws` workspace file
Save Workspace As	Save the current workspace using a new file dialog every time
Open Workspace	Close all open tabs and reopen the set of files stored in a `.vaws` workspace file
Set working folder	Choose the default folder used by file pickers and save dialogs. The path is stored in the app config, and menu previews keep the end of the path visible.
Open Project (`Menu → File`)	Open a multi-file `.proj` project and open all source files as tabs
Save Project (`Menu → File`)	Save the current `.proj` project (project panel must be open)
Close Project (`Menu → File`)	Close the currently open project and all its file tabs. Prompts to save unsaved changes. The project panel resets to its empty state.
Load .asm file	Opens a `.asm` file in Expert mode and imports textual 6502 ASM into the current tab
Save PRG	Export the compiled binary as a `.prg` file
Build CRT	Export the program as a 64K Magic Desk (`.crt` , cartridge type 19) file. See Section 12b.
Run (split button)	The main ▶ **Run** button runs the current mode; click the ▾ arrow to switch between: **Run as PRG** (compile and launch VICE directly), **Run via D64** (package into a .d64 disk image and launch VICE), or **Run on hardware** (send PRG to a C64 Ultimate / 1541 Ultimate device). See Section 12 and Section 13.
Debug (RetroDebugger)	Compile and launch in RetroDebugger with breakpoints, symbols, and autostart flags (see Section 9)
Run with Exomizer	Checkbox in the Settings menu — when enabled, all Run and Build operations crunch the PRG through `exomizer sfx sys` before launching or saving. Works with Run as PRG, Run via D64, Run on Hardware, Build PRG, and Build D64. Configure the Exomizer executable in **Hardware Settings** first.
Automatic snapshot saving	Checkbox in **Hardware Settings** → **Snapshot**. When enabled, the app creates a snapshot automatically about 2.5 seconds after you stop editing a tab. Turn it off if you only want manual snapshot saves.
Hardware Settings	Open the hardware configuration dialog — configure VICE, Exomizer, RetroDebugger, and C64 Ultimate (host, password, connection test). See Section 13.
New program...	Opens a confirmation dialog, then clears all blocks from the program area
Collapse All	Collapse all blocks
About	Version info
What's New	Changelog

Project snapshots

Project snapshots are stored as on-disk sidecar JSON files, not in localStorage. They are tied to the current project file when one exists, so the history survives restarts and follows the project around.

- **Menu** → **Build** → **Save snapshot** opens the snapshot dialog and saves the current block state plus Expert ASM text.
- **Menu** → **Build** → **Restore previous version** restores the latest snapshot directly.
- **Menu** → **Build** → **Snapshot history** opens the dialog where you can add notes, restore older entries, or delete them.
- **Hardware Settings** → **Snapshot** → **Automatic snapshot saving** controls whether the app creates snapshots automatically after edits. The default delay is about 2.5 seconds, and the setting applies per tab.
- If a project has not been saved yet, snapshots are stored in the app config directory until the project gets a file path.
  | **Knowledge Base** | Reference links (6502 opcodes, C64 KERNAL, memory map, colors) |
  | **Check for Update** | Open the itch.io page to check for a newer release |

Workspaces

A **workspace** (`.vaws` file) remembers which real, on-disk files were open across every tab — including each tab's editor mode and which tab was active — so you can reopen that exact set later. It is separate from a `.proj` project: a workspace can span any mix of Block/Expert `.json` project files, standalone `.asm` files, and Ultimate Basic `.ub` / `.proj` sources across multiple tabs.

- Workspaces **auto-save** a few hundred milliseconds after you make a change once one has been saved or opened.
- The app **auto-restores your last workspace** on launch, so your open tabs pick up where you left off.
- Only tabs backed by a real file on disk are saved into the workspace; a tab holding an unsaved sample or an in-memory-only program has nothing to persist and is skipped (with a notice if none of the open tabs qualify).
- Opening a workspace closes all currently open tabs first — you are asked to confirm before it proceeds.
- If a workspace references a file that has since been moved or deleted, that entry is skipped and reported by name after loading.

Load .asm file (quick reference)

The Expert-mode `.asm` loader accepts common 6502 source patterns and converts them into blocks:

- `* = $1500` → ORG block
- `Label:` → LABEL block
- `Label: .byte 0` → LABEL + BYTE blocks
- `.byte ...` → BYTE block
- `; comment` (or inline `; ...`) → COMMENT block
- instructions (`lda` , `jsr` , `beq` , etc.) → instruction blocks with detected addressing mode

Import parsing notes and best practices

- Local labels like `.wait` are imported as standard labels (dot removed), and references are normalized accordingly.
- For `($zp),Y` / `($zp,X)` style addressing, use a concrete zero-page byte (`$FB` , `$FC` , etc.) for best compatibility.
- Avoid ambiguous short labels that look like hex (`cc1` , `dead` , `beef`) in branch contexts; prefer names like `loop_cc1` .
- If your program starts with data (`.byte`) before executable code, add an explicit entry jump (for example `JMP Start`) at the top.

ASM import (Kick Assembler)

The **ASM import** button on the Program menu ingests raw Kick Assembler source into a new block-mode tab. It is separate from the Expert-mode `Load .asm file` above — its custom tooltip flags that **only Kick Assembler code is supported** (other assemblers may parse partially but are not guaranteed to round-trip).

Supported patterns:

- `.pc = $XXXX` origin directive → ORG block
- `.const NAME = value`, `.label NAME = value` → CONST equate
- `.macro NAME(p1, p2, ...) { ... }` with `{` / `}` brace body or `.endm` → user macro definition
- Macro invocation `NAME(args)`, Kick colon prefix `:NAME(args)`, and `.invoke NAME(args)` — all round-trip via the Kick colon form
- `@local` labels (`@loop:`, `BEQ @loop`) preserve the `@` prefix as-is
- Operand `label + N` / `label - N` (e.g. `STA mod1+2`, `LDA xp+1`)
- Line comments `// ...` and `;` — both accepted, `/* ... */` blocks are treated as a single comment line
- BASIC autostart passthrough: when the program starts at `$0801` with the standard `SYS 2061` byte stub (`.byte $0B,$08,$0A,$00,$9E,$32,$30,$36,$31,$00,$00,$00`), the compiler emits the PRG verbatim instead of wrapping a second BASIC SYS around it

Known limitation:

- Constants that resolve to a zero-page address (for example `.const BYTEADDR = $FC` used as `STA BYTEADDR`) currently compile to absolute-mode instructions (3 bytes) instead of zero-page (2 bytes). The compiled code still writes to the correct memory location, just with a small size and cycle overhead compared to the same source built by Kick Assembler.

UltimateBasic Mode

Visual Assembler includes a complete **Ultimate Basic IDE**. Ultimate Basic is a modern compiled BASIC language for creating C64 programs, games, and demos without writing every operation in low-level 6502 assembly. The compiler runs locally and generates native C64 PRG output.

Opening the UB editor

Select the **UB** icon in the main toolbar to switch to Ultimate Basic mode. The selected editor mode is remembered across application restarts. A new source begins with:

```
color bg 0
color border 0

print "HELLO FROM ULTIMATE BASIC"
```

The UB mode works with `.ub` source files. **New**, **Open**, **Save**, and **Save As** operate on the active UB tab. Opening a `.ub` file activates the matching editor tab automatically.

The toolbar shows the current UB working folder. This folder is stored separately from the Block/Expert working folder. While UB mode is active, **File** → **Set working folder** selects the UB folder; its tooltip identifies the active scope. UB Open/Save dialogs start there, and unsaved sources use it as the base for relative `include` and `incbin` paths.

Editor tools

The UB toolbar follows the same visual language and custom tooltips as Expert mode. It provides:

- syntax highlighting based on the current Ultimate Basic language reference;
- line numbers that remain synchronized with long files;
- a minimap and editor zoom controls; click the minimap to jump or drag its viewport selection for continuous scrolling;
- Find (`Ctrl+F` / `Cmd+F`) using the Expert-style search bar;
- source formatting with structure-aware indentation;
- autocomplete for commands and built-in functions;
- a searchable **Commands** panel with syntax, description, and usage guidance — descriptions follow the current UI language (Hungarian, English, Spanish, German, Dutch), falling back to English for anything not yet translated;
- independently toggleable **Project** and **Commands** panels, displayed side by side when both are enabled;
- independently toggleable and resizable **Build Output** and **Disassembly** panels.

The Disassembly panel includes a **Copy** button that copies its complete displayed source to the clipboard. Command help follows the bundled compiler: for example, `sprite_frame id, data_address [, frame]` selects an animation image from consecutive 64-byte sprite frames.

The command list is intentionally height-limited so the command-detail card can fill the remaining panel height. The detail area scrolls independently for longer syntax descriptions.

Projects, tabs, and startup files

Ultimate Basic projects use `.proj` files and can contain multiple `.ub` sources. The Project panel lists open files, marks unsaved tabs, and shows discovered labels, functions, and subroutines. Project actions let you create, open, save, and close a project or add another source file.

Click the star beside a project file to mark it as the **startup file**. Build, Run, D64, C64 Ultimate, and Debug commands compile that startup source even if a different tab is currently active. Without a startup selection, the active UB tab is used.

Building and diagnostics

The **Build** button opens the same centered progress experience used by the other Visual Assembler run workflows. Successful builds update Build Output, Build Info, and Disassembly. Enable **Verbose** to include compiler memory-map details, internal zero-page allocations, and generated code data.

When compilation fails:

- Build Output is made visible automatically;
- compiler errors are rendered in red;
- the centered compile dialog displays the failure;
- errors containing a source line select that line in the active UB editor.

Build Info reports load/end addresses, code and PRG sizes, Exomizer status, variables, arrays, functions/subroutines, and labels.

Running, D64, and Exomizer

The main split **Run** button supports Ultimate Basic in every normal destination:

Run mode	Ultimate Basic behavior
Run as PRG	Compile and launch the PRG directly in VICE.
Run via D64	Compile, open the standard D64 packaging dialog, then launch the disk in VICE.
Run on Ultimate	Upload and run the PRG through the configured C64 Ultimate REST connection.
Run D64 on hardware	Package a D64 and send it to the configured C64 Ultimate.

The global **Settings** → **Exomizer** option also applies to UB builds and normal run targets; no separate UB toolbar toggle is needed. Debugger launches deliberately use the uncompressed PRG so compiler addresses and symbols continue to match the executed program.

Enable **Settings** → **Program Settings** → **Generate UltimateBasic ASM source (.asm)** to save the compiler-generated assembly beside a PRG or D64 build using the same base filename. This is a build option, so the UB toolbar does not contain separate ASM export buttons. A `load "NAME", $address` statement also supplies the matching D64 extra file's PRG load address.

Debugger symbols and disassembly

Builds request Ultimate Basic debug information and produce three compatible sidecars:

- `.sym` for KickAssembler-style symbols;
- `.dbg` for C64Debugger/RetroDebugger source and segment information;
- `.vs` for VICE monitor labels.

The colorized UB Disassembly panel resolves known labels and presents addresses, bytes, mnemonics, and operands. The **Debug** button launches RetroDebugger with the raw UB PRG, debug sidecars, and the compiler's labels, functions, subroutines, variables, and arrays. Debug wait and unpause settings are shared with the normal Visual Assembler debugger configuration.

Ultimate Basic manual and source

The book icon in the UB toolbar opens the matching Ultimate Basic `MANUAL.pdf` offline; the manual button in the startup welcome dialog opens the same manual. Visual Assembler takes both the compiler and PDF from the pinned upstream Git/Cargo dependency, so the IDE does not maintain a second copy of the Ultimate Basic implementation. The About dialog and splash screen display the actual dependency version.

Ultimate Basic is also available as a standalone open-source project:

https://github.com/zstarczali/UltimateBasic

The compiler is bundled into Visual Assembler, so no separate `ub` executable is required at runtime.

6. Expert Mode

Expert Mode is a full-featured direct-text 6502 assembly editor that lives alongside the block editor. Every tab can be in either Block mode or Expert mode — you switch between them freely at any time using the **Block / Expert** toggle in the top bar.

Switching modes

- **Block → Expert:** the current program is serialised to text (one instruction per line, labels, macros as directives). Edits in Expert mode are synced back to the block array whenever you switch back or trigger an action.
- **Expert → Block:** the text is parsed with `parseAsmText()` and the result replaces the block program. A compile-error dialog is shown if parsing fails.
- **Blank lines** are preserved through round-trips: empty lines in the Expert editor appear as thin dashed spacers in Block mode and are restored as empty lines when switching back to Expert.

Editor layout

```
┌──────────────────────────────────────────────────┐
│ [toolbar]  Block | Expert < tab toggle           │
├────────────┬─────────────────────────┬───────────┤
│  Palette   │   ASM text editor       │  Disasm   │
│  (opt.)    │   (monospace, editable) │  panel    │
│            │                         │  (opt.)   │
└────────────┴─────────────────────────┴───────────┘
```

Panel	Toggle	Description
Palette	`#expert-palette-btn`	The left block palette — drag blocks into the editor or click to insert at cursor
ASM editor	always visible	Full monospace textarea with live syntax highlight overlay
Disasm panel	`#expert-disasm-btn`	Pure 6502 disassembly: each instruction shows address, hex bytes, and numeric operands; macros fully expanded

Toolbar buttons

Button	ID	Function
Format	`#expert-format-btn`	Auto-format source (labels to col 0, 4-space indent, 1-space mnemonic/operand)
Load .asm	`#expert-load-asm-btn`	Open a `.asm` file — the content is loaded into a **new tab** with the filename as the tab label. Each loaded file becomes an independent tab with its own program blocks and editor state.
Save .asm	`#expert-save-asm-btn`	Save editor content to a `.asm` file (file dialog on first save)
Build Info	`#expert-build-info-btn`	Open the Build Info dialog (origin, size, labels, errors)
HL	`#expert-hl-btn`	Toggle syntax highlighting (disable for very large files)
Autocomplete	`#expert-autocomplete-btn`	Toggle expert autocomplete suggestions on/off. When disabled, no directive, mnemonic, or label popup appears in the expert editor.
Region selection	`#expert-region-selection-btn`	Toggle the automatic region highlight in Expert mode. The fold state stays stored, but when this is off the editor keeps the full source visible and does not auto-select the current region.

Button	ID	Function
Collapse / expand all regions	`#expert-region-fold-all-btn`	Fold or unfold every `.region` block in one click. If any region is currently open, the button collapses them all; if every region is already collapsed, the next click expands them all. The button lights up when all regions are collapsed.
Line numbers	`#expert-line-numbers-btn`	Toggle the line number gutter on the left side of the editor. The gutter stays in sync with the scroll position and updates live as you type.
Find	`#expert-find-btn`	Open the floating Find bar (`Ctrl+F`). Type to search; matches are highlighted in the overlay. `Enter` / `Shift+Enter` navigate between matches. `Escape` closes the bar.
Zoom out / in	`#expert-zoom-out-btn` / `#expert-zoom-in-btn`	Decrease / increase the editor font size (8–28 px). The setting is persisted.
Palette	`#expert-palette-btn`	Show/hide the left mnemonic palette
Disasm	`#expert-disasm-btn`	Show/hide the disassembly panel (pure 6502, macros expanded)
Monitor	`#expert-monitor-btn`	Show/hide the monitor hex-dump panel
Minimap	`#expert-minimap-btn`	Show/hide the code minimap strip on the right side of the editor

Editor shortcuts: `Ctrl+/` (`Cmd+/` on macOS) comments the current line or every selected line; adding `Shift` removes the leading comment marker from those lines.
Inline comments (for example `LDA $12 ; explanation`) remain on the instruction line when switching between Expert and Block mode. Block mode shows them as a green italic `; comment` marker in the block header; hovering the marker reveals the full text when it is truncated.

Expert editor minimap

The Expert editor minimap is a narrow canvas strip (`88 px`) on the far right of the editor area. It renders a scaled-down representation of every source line:

Bar colour	Token type
Comment colour	Lines starting with `;`
Label colour	Lines with a `label:` definition
Directive colour	`.byte` , `.macro` , `.region` , and all other directives
Mnemonic colour	Everything else (instructions)

A **semi-transparent viewport indicator** (accent-coloured rectangle) shows which part of the source is currently visible. Click anywhere on the minimap to jump to that position; drag to scroll continuously. The minimap scrolls independently to keep the viewport indicator centred. The state is persisted in UI settings (`expertMinimap` key).

Error highlighting

Lines that fail to compile are highlighted in **red** (tinted background + left accent border) in real time, 350 ms after each keystroke. The first error message is also shown in the status bar. Fix the line and the highlight disappears automatically.

Syntax highlight

The editor uses a transparent `<div>` overlay (`expert-hl`) that mirrors the textarea content with coloured `<span>` elements. Highlight can be toggled off with the **HL** button for performance on very large programs.

Colour	Token
Yellow-green	Mnemonics (`LDA`, `STA`, `JMP`, …)
Blue	Directives (`.byte`, `.word`, `.fill`, `*=`, …)
Orange	Numbers (`$FF`, `%1010`, `255`)
Cyan	Labels (lines ending in `:`)
Teal	String literals
Dark green	Comments (`; …`)

`REGION` / `ENDREGION` directives are highlighted like the other assembler directives. Collapsed regions keep only the region header visible in the editor until you reopen them. The new **Region selection** toolbar toggle only controls the automatic current-region highlight in Expert mode; turning it off keeps the source visible without changing the fold state.

Source formatter

Click the **Format** button (`#expert-format-btn`) to auto-format the current source:

- Label definitions are moved to column 0.
- Instructions are indented with 4 spaces.
- Mnemonics are uppercased.
- Exactly one space between mnemonic and operand (extra whitespace is normalised).
- If the source is already formatted, a `"Already formatted"` status is shown.

Project panel & tabs

Expert mode supports a **project panel** (`#expert-project-panel`) for multi-file `.proj` projects:

- A `.proj` file is a JSON manifest that lists source files and their metadata.
- Open a project with **Menu** → **File** → **Open project** or drag a `.proj` file onto the window.
- Each file in the project opens as a separate **tab** in the tab bar at the top of the editor.
- **Close Project** (`Menu → File → Close project` / `#menu-close-project`) closes the current project and all its file tabs at once. Prompts to save any unsaved changes before closing. The project panel resets to its empty state and `_expertProjectData` is cleared.
- Each file can be marked as the **startup file** (★ star icon). When a startup file is set, the **Run** button (PRG, D64, Ultimate) always assembles and runs that file's code — regardless of which tab is currently active. This works in both block mode and Expert mode.
- The **symbols** section at the bottom of the project panel can be resized vertically with the divider between the file tree and the symbols list, so long symbol lists can take more space when needed.

Tab bar

The tab bar appears above the editor when there is more than one tab open.

Feature	Description
Dirty dot	A small accent-coloured dot on the tab name indicates unsaved changes

Feature	Description
Scroll arrows	Left/right scroll buttons appear when there are more tabs than fit the bar
Close (×)	Closes the tab; prompts to save if the tab is dirty
File extension	The full filename including extension (`.c64va` , `.json`) is shown

> **Tip:** Palette sync (`#expert-palette-sync-btn`) keeps the palette selection in sync with the mnemonic at the cursor. Disable it when you prefer not to have the palette jump around as you edit.

---

7. Addressing Modes

Each 6502 instruction supports one or more addressing modes. The mode selector appears on each block.

Mode	Label	Example	Description
implied	Implied	`NOP`	No operand; the instruction is self-contained
immediate	Immediate	`LDA #$FF`	Inline constant; the assembler adds `#` automatically
zeroPage	Zero page	`LDA $10`	Single byte address in page zero (0–255)
zeroPageX	Zero page,X	`LDA $10,X`	Zero page address + X register offset (result wraps in page 0)
zeroPageY	Zero page,Y	`LDX $FB,Y`	Zero page address + Y register offset
absolute	Absolute	`LDA $0400`	Full 16-bit memory address
absoluteX	Absolute,X	`LDA $0400,X`	16-bit address + X register offset
absoluteY	Absolute,Y	`LDA $0400,Y`	16-bit address + Y register offset
relative	Relative/Label	`BNE loop`	For branch instructions; enter a label name or target address
indirectX	Indirect,X	`LDA ($FB,X)`	Zero page indexed indirect (operand = zero page address, 1 byte)
indirectY	Indirect,Y	`LDA ($FB),Y`	Zero page indirect indexed (operand = zero page address, 1 byte)
indirect	Indirect	`JMP ($0100)`	Indirect; only usable with JMP

Label expressions as operands

Any operand field that accepts an address or immediate value also accepts a **constant name** (from a `CONST` block or a `LABEL`) directly. Additionally, you can use **label+offset** or **label−offset** expressions to reference an address relative to a named constant:

Syntax	Example	Description
`label`	`STA screen_ram,X`	Resolves to the label/constant value
`label+$hex`	`STA screen_ram+$0100,X`	Label address plus a hex offset
`label+decimal`	`STA screen_ram+256,X`	Label address plus a decimal offset

Syntax	Example	Description
`label-$hex`	`LDA table-$10`	Label address minus a hex offset
`#<label`	`LDA #<screen_ram`	Low byte of the label address
`#>label`	`LDA #>screen_ram`	High byte of the label address
`*`	`BNE *`	Current program counter (the instruction's own address); branches with `*` generate an infinite self-loop (offset `$FE`)

Example — clear two screen pages using a CONST:

```
; .CONST screen_ram = $0400
    LDX #$00
clear:
    STA screen_ram,X
    STA screen_ram+$0100,X
    DEX
    BNE clear
```

---

8. Standard 6502 Instructions

Data Movement

Mnemonic	Description	Modes
`LDA`	Load Accumulator	immediate, zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`LDX`	Load X register	immediate, zeroPage, zeroPageY, absolute, absoluteY
`LDY`	Load Y register	immediate, zeroPage, absolute, absoluteX
`STA`	Store Accumulator	zeroPage, absolute, absoluteX, absoluteY, indirectX, indirectY
`STX`	Store X register	zeroPage, zeroPageY, absolute
`STY`	Store Y register	zeroPage, absolute

Arithmetic

Mnemonic	Description	Notes
`ADC`	Add with carry	Set carry with `SEC` before use in most cases
`SBC`	Subtract with carry	Set carry with `SEC` before subtraction
`INC`	Increment memory	—

Mnemonic	Description	Notes
`DEC`	Decrement memory	—
`CMP`	Compare with A	Sets flags; does not modify A
`CPX`	Compare with X	—
`CPY`	Compare with Y	—

Logic

Mnemonic	Description
`AND`	Logical AND with Accumulator
`ORA`	Logical OR with Accumulator
`EOR`	Exclusive OR with Accumulator
`BIT`	Test bits in memory against A (sets N, V, Z flags)

Jumps & Branches

Mnemonic	Description
`JMP`	Unconditional jump (absolute or indirect)
`JSR`	Jump to subroutine (saves return address on stack)
`RTS`	Return from subroutine
`RTI`	Return from interrupt
`BNE`	Branch if Not Equal (Z=0)
`BEQ`	Branch if Equal (Z=1)
`BCC`	Branch if Carry Clear (C=0)
`BCS`	Branch if Carry Set (C=1)
`BMI`	Branch if Minus (N=1)
`BPL`	Branch if Plus (N=0)
`BVC`	Branch if Overflow Clear (V=0)
`BVS`	Branch if Overflow Set (V=1)

Register Operations

Mnemonic	Description
`TAX`	Transfer A → X
`TAY`	Transfer A → Y
`TXA`	Transfer X → A

Mnemonic	Description
`TYA`	Transfer Y → A
`TSX`	Transfer Stack pointer → X
`TXS`	Transfer X → Stack pointer
`INX`	Increment X
`DEX`	Decrement X
`INY`	Increment Y
`DEY`	Decrement Y

Shift & Rotate

Mnemonic	Description
`ASL`	Arithmetic Shift Left
`LSR`	Logical Shift Right
`ROL`	Rotate Left through Carry
`ROR`	Rotate Right through Carry

Stack

Mnemonic	Description
`PHA`	Push Accumulator onto stack
`PHP`	Push Processor status onto stack
`PLA`	Pull Accumulator from stack
`PLP`	Pull Processor status from stack

System / Flags

Mnemonic	Description
`CLC`	Clear Carry flag
`CLD`	Clear Decimal mode
`CLI`	Clear Interrupt disable
`CLV`	Clear Overflow flag
`SEC`	Set Carry flag
`SED`	Set Decimal mode
`SEI`	Set Interrupt disable
`NOP`	No operation

Mnemonic	Description
`BRK`	Force break / software interrupt

Illegal / Undocumented Instructions

These are supported for advanced use. Use with care — behavior may differ between chips.

`LAX` , `SAX` , `DCP` , `ISC` , `SLO` , `RLA` , `SRE` , `RRA` , `ANC` , `ALR` , `ARR` , `AXS`

---

9. Macro Blocks — Reference

Macro blocks let you do common tasks in one step — instead of writing 10–20 instructions by hand, you drop one block and the assembler generates the code for you. Think of them as built-in subroutines.

LABEL

Like a **line number in BASIC** — but with a name instead of a number. Jump targets for `JMP` , `JSR` , `BNE` , etc.

Field	Description
Label name	Identifier used in `JMP` , `JSR` , `BNE` , etc.

Expert syntax:

```
loop:
```

Generated ASM:

```
loop:  ; $0820
```

The current address is shown as a comment. Labels have **0 byte size**.

COMMENT

Like **REM in BASIC** — a note for yourself that the assembler ignores completely.

Expert syntax:

```
; Your comment text here
```

Generated ASM:

```
; Your comment text here
```

BYTE

Like **DATA in BASIC** — stores a list of raw byte values inline in the program.

Field	Description
Operand	Comma-separated byte values (e.g. `$01, $02, $FF` or `1, 2, 255`)

Expert syntax:

```
.byte $01, $02, $FF
```

Generated ASM:

```
    .byte $01, $02, $FF
```

Lo/hi byte label references: BYTE accepts KickAssembler / ca65 style `<label` (low byte) and `>label` (high byte) tokens alongside numeric values. The assembler resolves the label address at compile time and inserts the appropriate byte. Example:

```
    .byte <frame_0, >frame_0, <frame_1, >frame_1
```

This stores the low byte of `frame_0`'s address, then the high byte, then the same for `frame_1`. Useful for building jump tables and address lists.

Size: Number of bytes in the list.

WORD

Like **DATA in BASIC but for 16-bit numbers**. Each value is stored as two bytes (low byte first, then high byte — 6502 little-endian order).

Field	Description
Operand	Comma-separated 16-bit values (e.g. `$0400, $C000`)

Expert syntax:

```
.word $0400, $C000
```

Generated ASM:

```
    .word $0400, $C000
```

Size: 2 bytes per word.

FILL

Like `FOR I=1 TO N : POKE addr+I, val : NEXT` — fills a block of memory with the same byte, but in a single block. Great for clearing areas or pre-filling tables.

Field	Description
Operand	`count,value` — e.g. `256,0` fills 256 bytes with zero

Expert syntax:

```
.fill 256, $00
```

Generated ASM:

```
.fill 256, $00
```

Expression syntax: Both `count` and `value` accept arithmetic expressions. You can reference CONST names, use hex/binary literals, and call built-in math functions:

Expression	Meaning
`TILE_COUNT, $00`	count from a CONST, value hex literal
`40*25, 0`	inline multiplication
`round(sin(PI/4)*255), $80`	trigonometry

Built-in functions: `sin()`, `cos()`, `round()`, `max(a,b)`, `min(a,b)`, `abs()`, constant `PI`

Operators: `+ - * /` Literals: `$FF` (hex), `%10110000` (binary) Low/high byte: `lo(expr)`, `hi(expr)`

Size: The count value in bytes.

ALIGN

Slides the current address forward to the next clean boundary by inserting zero-padding bytes. The C64 requires sprite data to start on a 64-byte boundary — `ALIGN 64` handles that automatically.

Field	Description
Boundary	Alignment value — e.g. `64` (sprite boundary), `256` (page), `$2000` (bitmap)

Expert syntax:

```
.align 64
.align $2000
```

Generated ASM:

```
; ALIGN 64 → $0840 (12 bytes padding)
```

Size: Dynamic — depends on the current program counter position.

> **Tip:** Use `ALIGN 64` before sprite data, `ALIGN 256` to ensure page-aligned tables.

TEXT

Like **PRINT AT** — writes text directly to the C64 screen at a given column and row, without using the KERNAL. It generates one LDA/STA pair per character, targeting screen RAM at `$0400`.

Field	Description
Text	The string to display
X	Column (0–39)
Y	Row (0–24)
Label (optional)	Assigns a label pointing to the computed screen address
Lowercase charset	Checkbox — see below

Charset modes:

The C64 has two character sets selectable at runtime:

Mode	$D018 bit 1	Uppercase input	Lowercase input
Uppercase/graphics (default)	0	`A` – `Z` → screen codes $01–$1A ✓	treated as uppercase too
Lowercase/uppercase (after CHARSET lower)	1	`A` – `Z` → $01–$1A (uppercase)	`a` – `z` → $41–$5A (lowercase) ✓

- **Uppercase charset (default, checkbox unchecked):** Type what you want to see in uppercase. `"HELLO"` displays as `HELLO`. Lowercase input is mapped to uppercase screen codes.
- **Lowercase charset (checkbox checked):** Type the exact case you want to see. `"hello"` → lowercase display, `"HELLO"` → uppercase display. Requires a runtime charset switch before the screen is written (use the **CHARSET lower** macro).

Generated ASM (uppercase mode, `"HELLO"`):

```
LDA #$08      ; 'H' screen code $08
STA $0400
LDA #$05      ; 'E' screen code $05
STA $0401
...
```

Expert syntax:

```
.text 0, 2, "HELLO"           ; uppercase charset (default)
.text 0, 2, "hello", lower    ; lowercase charset
```

Characters are encoded as **screen codes** (not PETSCII). **Size:** `text.length × 5` bytes (LDA + STA per character).

STRING

Like **POKEing a string** into any memory address at runtime. Generates LDA/STA pairs that copy each character's screen code to consecutive addresses.

Field	Description
Text	The string to write
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address
Shift	Hex value (00–FF) added to each screen code byte (e.g. `$80` = reverse video)
Lowercase charset	Checkbox — same semantics as TEXT (see TEXT section)

Expert syntax:

```
.string $C000, "HELLO"                ; uppercase charset (default)
.string $C000, "hello", lower         ; lowercase charset
.string $C000, "HELLO", 80            ; with shift (reverse video)
.string $C000, "hello", 80, lower     ; shift + lowercase
.string $C000, "HELLO" :my_string     ; with macroLabel
```

Generated ASM:

```
    LDA #$08      ; 'H' screen code
    STA $C000
    LDA #$05      ; 'E' screen code
    STA $C001
    ...
```

Characters are encoded as **screen codes** (not PETSCII). The optional **Shift** value is added to every byte, e.g. `$80` for reverse video. **Size:** `text.length × 5` bytes (one LDA + one STA per character).

DATA

Like a **POKE loop** — writes a list of raw bytes to a memory address at runtime, one LDA/STA pair per byte.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address

Expert syntax:

```
.data $C000, $01, $02, $03          ; hex address
.data my_buf, $01, $02, $03         ; label address
.data $C000, $01, $02, $03 :mydata  ; with macroLabel
```

Generated ASM:

```
LDA #$01
STA $C000
LDA #$02
STA $C001
...
```

Size: `byte_count × 5` bytes (one LDA + one STA per byte).

RAWBYTES

Like **DATA that loads directly into memory** — no runtime code at all. The bytes are present from the moment the PRG loads, before your code even starts. Use this for sprite data, level maps, lookup tables, anything that just needs to be at a specific address.

Field	Description
Bytes	Comma-separated byte values
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address

Expert syntax:

```
.rawbytes $C000, $00, $00, $00      ; hex address
.rawbytes sprite_data, $00, $00     ; label address
```

```
.rawbytes $0C50, $00, $00 :nev      ; with macroLabel — other code can use LDA nev,X
```

Size in code: 0 bytes. The data is placed at the given address in the output.

> **DATA vs RAWBYTES:** DATA generates LDA/STA code that copies bytes at runtime (slower, but works if the data needs to be dynamic). RAWBYTES just places the bytes directly — no code, instant, zero cost.

RAWTEXT

Like RAWBYTES but for text — encodes the string as screen codes and places the bytes at a fixed address with **no runtime code**. The text is ready in memory the instant the PRG loads.

Field	Description
Text	String to encode
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address
Shift	Hex value (00–FF) added to each screen code byte (e.g. `$80` = reverse video)
Lowercase charset	Checkbox — same semantics as TEXT (see TEXT section)

Expert syntax:

```
.rawtext $C000, "HELLO"                ; uppercase charset (default)
.rawtext $C000, "hello", lower         ; lowercase charset
.rawtext $C000, "HELLO", 80            ; with shift (reverse video)
.rawtext $C000, "hello", 80, lower     ; shift + lowercase
.rawtext $0400, "HELLO" :my_text       ; with macroLabel
```

Generated ASM:

```
; .rawtext "HELLO" -> $C000
; $C000
    .byte $08, $05, $0C, $0C, $0F   ; H E L L O (uppercase screen codes)

; .rawtext "hello", lower -> $C000
; $C000
    .byte $48, $45, $4C, $4C, $4F   ; h e l l o (lowercase screen codes $41–$5A range)
```

Size in code: 0 bytes. The data is placed at the given address in the output.

> **STRING vs RAWTEXT:** STRING generates LDA/STA code that copies the text at runtime. RAWTEXT bakes the bytes into the PRG at load time — no code, no waiting.

PETSCII

Like **RAWBYTES but for KERNAL output** — encodes the string as PETSCII bytes (compatible with CHROUT at `$FFD2`) and places them at a fixed address with no runtime code. Use this when you want to print characters via `JSR $FFD2` in a loop, and note that the new `PRINT` macro uses the same encoder and lowercase checkbox behavior.

> **PETSCII vs screen codes:** PETSCII and screen codes are two different encodings. Screen code `$01` = letter A; PETSCII `$41` = letter A (via CHROUT). Use PETSCII only when printing through the KERNAL; use TEXT/STRING/RAWTEXT for writing directly to screen RAM.

Field	Description
Text	String to encode as PETSCII bytes
Address	Target memory address — `$C000` hex or a **label name**
Label (optional)	Assigns a label pointing to the target address
Lowercase PETSCII	Checkbox — see below

Charset modes:

Mode	Uppercase input (`A – Z`)	Lowercase input (`a – z`)
Uppercase (default, unchecked)	`$41` – `$5A` (PETSCII uppercase via CHROUT)	also mapped to `$41` – `$5A`
Lowercase (checked)	Alphabetic letters are remapped so the visible case stays consistent on the lowercase/uppercase charset	Same rule

Expert syntax:

```
.petscii $C000, "HELLO"              ; uppercase PETSCII (default)
.petscii $C000, "hello", lower       ; lowercase PETSCII ($61–$7A)
.petscii $C000, "HELLO", null        ; with null terminator
.petscii $C000, "hello", lower, null ; lowercase + null terminator
.petscii $C000, "HELLO" :my_msg      ; with macroLabel
```

Generated bytes (uppercase, `"HELLO"`):

```
; $C000
    .byte $48, $45, $4C, $4C, $4F   ; H E L L O (PETSCII $41–$5A range)
```

Size in code: 0 bytes. The data is placed at the target address as a deferred data section (like RAWBYTES).

Null terminator: Check the "*Append `$00` (null terminator)*" checkbox to automatically add a `$00` byte after the text. Ideal for null-terminated loops:

```
    LDX #$00
loop:
    LDA msg,X
    BEQ done        ; $00 stops the loop
    JSR $FFD2
    INX
    BNE loop
done:
    RTS
```

Encoding rules:

Input	Uppercase mode	Lowercase mode
`A` – `Z`	`$41` – `$5A`	`$61` – `$7A`
`a` – `z`	`$41` – `$5A` (forced uppercase)	`$41` – `$5A`
Space, digits, punctuation (32–126)	as-is	as-is
Newline	`$0D` (RETURN)	`$0D`
Other	`$20` (space)	`$20`

> **Tip:** Use PETSCII for data that will be output via CHROUT (`$FFD2`). For writing directly to screen RAM, use STRING or RAWTEXT instead.

CHARSET

Switches the VIC-II character ROM between uppercase/graphics mode (C64 default) and lowercase/uppercase mode, by modifying bit 1 of `$D018` at runtime.

Field	Description
Mode	**Lowercase** — enables the lowercase/uppercase charset; **Uppercase** — restores the default uppercase/graphics charset

Expert syntax:

```
.charset lower    ; switch to lowercase charset
.charset upper    ; switch back to uppercase/graphics charset
```

Generated ASM:

Lowercase mode:

```
    LDA $D018
    ORA #$02      ; set bit 1 → lowercase/uppercase ROM at $1800
    STA $D018
```

Uppercase mode:

```
    LDA $D018
    AND #$FD      ; clear bit 1 → uppercase/graphics ROM at $1000
    STA $D018
```

Size: 8 bytes (LDA abs + ORA/AND imm + STA abs).

Why ORA/AND instead of a direct write? `$D018` also controls screen RAM location (bits 7–4). Toggling only bit 1 preserves the rest of the register.

Typical workflow:

```
    CHARSET lower             ; switch to lowercase charset
    TEXT 0, 0, "hello world"  ; [checkbox: Lowercase charset]
    ...
    CHARSET upper             ; restore default when done
```

Or in expert mode:

```
.charset lower
.text 0, 0, "hello world", lower
.charset upper
```

In Expert mode the `.charset` block now round-trips through the mode dropdown too, so the block preview and exported source stay aligned.

> **Note:** The CHARSET macro only changes the VIC character ROM pointer. It does not call `$E544` (KERNAL charset init). For most cases this is sufficient; call `JSR $E544` first only if you need the KERNAL's own print routines to respect the change.

CHARDEF

Defines a single 8×8 custom character in a RAM-based character set. Emits inline runtime code that copies 8 bytes into `base + index * 8` at execution time — no need for a preceding label or `ORG`.

Field	Description
Charset base	Base address of the RAM charset (default `$3800`). Must be aligned so the VIC-II can see it (see below).
Character index	Which character slot to redefine, 0–255. `65` = 'A' in the default screen code layout.
8 bytes	Comma-separated bitmap rows, top-to-bottom. Each byte's bit 7 = leftmost pixel.

Expert syntax:

```
.chardef $3800, 65, $18,$3C,$66,$7E,$66,$66,$66,$00
```

Generated ASM (8 × `LDA #b` / `STA target+n`, 40 bytes total):

```
    LDA #$18
    STA $3A08
    LDA #$3C
    STA $3A09
    LDA #$66
    STA $3A0A
    LDA #$7E
    STA $3A0B
    LDA #$66
    STA $3A0C
    LDA #$66
    STA $3A0D
    LDA #$66
    STA $3A0E
    LDA #$00
    STA $3A0F
```

Target address: `$3800 + 65 * 8 = $3A08`. Computed at compile time and hard-coded into the STA operands.

Size: 40 bytes per character (8 × 5).

Typical workflow:

```
; Point VIC-II at RAM charset at $3800
LDA $D018
AND #$F1
ORA #$0E       ; bits 3-1 = %111 → charset at bank+$3800
STA $D018

CHARDEF $3800, 65, $18,$3C,$66,$7E,$66,$66,$66,$00   ; redefine 'A'
CHARDEF $3800, 66, $7C,$66,$66,$7C,$66,$66,$7C,$00   ; redefine 'B'
; ... more CHARDEF calls for each custom char
```

When to use CHARDEF vs alternatives:

Approach	Use when
CHARDEF	You need a few custom characters (say 1–20). Runtime cost is 40 bytes each.
RAWBYTES @ $3800	You have a full custom charset (256 chars). Total 2 KB of data, no runtime copy.
INCBIN "charset.bin" @ $3800	External charset file (built by the Character Editor). Cleanest option.
Charset Canvas + INCBIN	Full 256-character bitmap painted as one 128×128 image.

> **Alignment reminder:** VIC-II expects the charset base at a multiple of `$0800`. Valid banks: `$0000`, `$0800`, `$1000`, … , `$3800` (within the current 16 KB VIC bank). RAM charsets typically live at `$2000`, `$2800`, `$3000` or `$3800`.

BOX_HIT

Axis-aligned bounding-box (AABB) collision test between two rectangles described by 4-byte zero-page structures. Returns the result in the accumulator: **A = 1** on overlap, **A = 0** otherwise. Pure inline assembly, no subroutine call.

Field	Description
Box1 ZP address	Zero-page base of the first box's 4-byte struct (default `$FB`).
Box2 ZP address	Zero-page base of the second box's 4-byte struct (default `$F7`).

Struct layout (4 bytes per box, unsigned 8-bit coordinates):

Offset	Field
`+0`	Left
`+1`	Top

Offset	Field
`+2`	Right
`+3`	Bottom

Expert syntax:

```
.box_hit $FB, $F7
```

Generated ASM (30 bytes, fully PC-relative — no subroutine, no absolute jumps):

```
    LDA $FD        ; b1 right
    CMP $F7        ; b2 left
    BCC no         ; R1 < L2 → miss
    LDA $F9        ; b2 right
    CMP $FB        ; b1 left
    BCC no
    LDA $FE        ; b1 bottom
    CMP $F8        ; b2 top
    BCC no
    LDA $FA        ; b2 bottom
    CMP $FC        ; b1 top
    BCC no
    LDA #$01       ; hit
    BNE done       ; unconditional (A ≠ 0)
no: LDA #$00
done:
```

Size: 30 bytes.

Typical workflow:

```
    ; Player sprite box at $FB..$FE
    LDA sprite_x            ; L
    STA $FB
    LDA sprite_y            ; T
    STA $FC
    CLC
    ADC #23                 ; +23 → B (24-pixel tall sprite)
    STA $FE
    LDA $FB
    CLC
    ADC #23                 ; R
```

```
    STA $FD

    ; Enemy box at $F7..$FA (populated similarly)
    ; ...

    BOX_HIT $FB, $F7        ; test player vs enemy → A = 0 or 1

    CMP #$01
    BNE no_collision
    JSR handle_hit
no_collision:
```

Constraints:

- Both zero-page addresses must be `≤ $FC` (each box needs 4 consecutive bytes: `zp`, `zp+1`, `zp+2`, `zp+3`).
- Coordinates are treated as **unsigned 8-bit** (0–255). For signed sprite coordinates outside this range, normalize before storing.
- The two boxes may overlap in ZP space if desired, but usually you want 8 distinct bytes.

Why not a subroutine? Inline generation avoids the JSR/RTS overhead (14+ cycles) and keeps the test hot in cache for tight game loops. If you need to test many pairs, put your own `JSR box_hit_sub` around a single BOX_HIT block manually.

Comparison with UB's `box_hit()` function: Ultimate Basic wraps the same 6502 logic as a runtime function returning to a variable. In VA, you place BOX_HIT inline where you need the test; the result is in `A`.

INCBIN

Like **BLOAD in BASIC** — picks up an external binary file (`.bin`, `.prg`, `.sid`, `.raw`) and embeds it directly into the assembled PRG at the address you specify.

Field	Description
File	Browse to select a `.bin`, `.prg`, `.sid`, or `.raw` file
Address	Target load address (e.g. `$C000`)

Expert syntax:

```
.incbin "music.bin", $C000
```

Generated ASM comment:

```
; INCBIN "music.bin" @ $C000 (2048 bytes)
.byte $01, $02, ...
```

Size in code: 0 bytes (deferred data section). The binary is embedded at the given address.

SID

Like **BLOAD for music** — loads a `.sid` file into your PRG and automatically reads its Init and Play addresses from the header. Call Init once at startup, then call Play from your IRQ handler every frame.

Field	Description
File	Browse to select a `.sid` file
Custom address (optional)	Override the SID's native load address (e.g. `$1000`). Leave empty to use the address from the SID header.

The block displays:

- **Title / Author** from the SID header
- **Load address** — where the data is placed in memory (effective address after any override)
- **Init address** — call this with JSR to initialize the music (adjusted for relocation if a custom address is used)
- **Play address** — call this with JSR on every frame in an IRQ handler (adjusted for relocation)
- A **(relocated)** badge appears when a custom address shifts the data from its original position

Expert syntax:

```
.sid "Ikari_Warriors.sid"
.sid "Ikari_Warriors.sid", $1000
```

Generated ASM comment:

```
; SID "Ikari_Warriors.sid" @ $1000  Init:$1000  Play:$1006  (4096 bytes)
```

Size in code: 0 bytes inline. The SID binary is placed at the specified address as a deferred chunk in the PRG.

> **Important:** Most SID files contain hardcoded internal absolute addresses. They can only be relocated if the entire binary is shifted by the same offset. If a SID has internal jumps to `$10xx`, it must remain at `$1000` — moving it to a different address will break those internal references.

> **Typical usage:** Place an ORG block before the SID block to set its address. Call Init once at startup, then call Play every frame from a raster IRQ handler.

INCLUDE

Like **MERGE in BASIC** — pulls in another file and expands its blocks inline at this position. Perfect for reusable subroutine libraries. The included blocks are read-only in the current project.

Two file kinds are supported:

- **Visual Assembler project** (`.json`) — the project's blocks are inserted as-is.
- **Plain assembly source** (`.inc` , `.asm` , `.s`) — the file is read as text and parsed the same way as expert mode. Every time you compile, the file is re-read from disk (source of truth = the file), so you can edit it externally with any editor.

Field	Description
File	Browse to select a `.json` project or a `.inc` / `.asm` / `.s` assembly source
Load address (optional)	If set (hex, e.g. `C000`), the included blocks are placed at that address — a synthetic `ORG` is injected before them, overriding any ORG block inside the included file. Leave empty to let the included file's own ORG blocks control placement.

Expert-mode syntax:

```
.include "library.json"
.include "macros.inc", $1500
.include "sprites.asm"
```

- The **file extension is required in expert mode** — a bare name like `.include "macros"` is treated as `.include "macros.json"` .
- Path resolution: first tries next to the project file (relative), then falls back to the app's bundled `samples/` directory.

Generated ASM (no address override):

```
; .include "library.json" — 12 block(s)
... (expanded blocks follow)
```

Generated ASM (with load address `C000`):

```
; .include "library.json" @ $C000 — 12 block(s)
*=$C000
... (expanded blocks follow)
```

> **Tip:** Use INCLUDE to build reusable subroutine libraries that you can share across projects. `.inc` / `.asm` / `.s` files are best when you want to edit the library in a plain text editor or share it with other 6502 assemblers; `.json` when the library is authored in Visual Assembler itself. Set a load address when the library has no ORG of its own, or when you want to override its default placement.

TABLE

Like **DIM at a specific address** — names a lookup table and sets where it lives in memory. Place BYTE, WORD, or FILL blocks after it to define the table's contents.

Field	Description
Name	Label identifier for the table (e.g. `color_table`)
Address	Fixed address where the table starts (e.g. `$C000`)

Expert syntax:

```
.table color_table, $C000
```

Generated ASM:

```
color_table:
```

The program counter jumps to the specified address. Place BYTE/WORD/FILL blocks after TABLE to fill the content.

Size: 0 bytes.

ORG

Sets where in memory the program (or a section of it) is placed — like choosing a start address before typing in machine code. Every program needs at least one ORG. The standard C64 BASIC-loadable start is `$0801`.

Field	Description
Address	The new origin address (e.g. `0801` in HEX, or `2049` in DEC)
HEX / DEC	Toggle the address input between hexadecimal and decimal display

Expert syntax:

```
* = $C000
```

Generated ASM:

```
* = $C000
```

Size: 0 bytes. The ORG block itself generates no machine code.

Each ORG block starts a new section. Blocks that follow are assembled starting at that address. When you export the PRG, all sections are merged into one file — gaps between sections are filled with zeros.

Example — code at `$0801`, data table at `$C000`:

```
* = $0801
    LDX #$00
loop:
    LDA $C000,X
    STA $D800,X
    INX
    BNE loop
    RTS

* = $C000
    .byte $01, $02, $03, ...
```

> **Tip:** Every program must start with an ORG block. The typical starting address for a C64 BASIC-loadable program is `$0801` (2049 decimal). When **BASIC SYS stub** is enabled, the assembler adds a short BASIC line at `$0801` and your code starts at `$080D`.

LOOP / NEXT

Like `FOR X=N TO 1 STEP -1 : ... : NEXT X` in BASIC — counts down from N to 1 using the X or Y register. Drop a LOOP block, put your instructions between it and NEXT, and it loops the right number of times automatically.

LOOP

Field	Description
Register	`X` or `Y` — the counter register
Count	Loop iteration count (hex or decimal, e.g. `0A` = 10)
Label	Auto-generated loop label (e.g. `loop0`)

Expert syntax:

```
.loop X, 10, loop0
```

Generated ASM:

```
    LDX #$0A
loop0:
```

Size: 2 bytes (LD_ opcode + immediate operand).

NEXT

Field	Description
Register	Automatically matched to the LOOP register
Label	Automatically linked to the LOOP label

Expert syntax:

```
.next loop0
```

Generated ASM:

```
    DEX
```

```
    BNE loop0
```

Size: 3 bytes (DEX + BNE + branch offset).

Example — clear 10 screen cells:

```
    LDX #$0A
loop0:
    LDA #$20        ; space character
    STA $0400,X
    DEX
    BNE loop0
```

FOR / ENDF

Like `FOR X=0 TO N-1 : ... : NEXT X` in BASIC — counts *up* from 0. Ideal when you need a forward index, e.g. stepping through a string or an array.

FOR

Field	Description
Register	`X` or `Y` — the counter register
Count	Loop limit (hex or decimal, e.g. `$12` = 18). X/Y runs from 0 up to limit-1.
Label	Auto-generated loop label (e.g. `for0`)

Expert syntax:

```
.for X, $12, for0
```

Generated ASM:

```
    LDX #$00
for0:
```

Size: 2 bytes (LD_ opcode + `#$00`).

ENDF

Field	Description
Register	Automatically matched to the FOR register
Label	Automatically linked to the FOR label
Count	Automatically copied from the paired FOR

Expert syntax:

```
.endf for0
```

Generated ASM:

```
    INX
    CPX #$12
    BNE for0
```

Size: 5 bytes (IN_ + CP_ #imm + BNE offset).

Example — print a null-terminated string:

```
    LDX #$00
for0:
    LDA msg,X       ; msg = PETSCII string at fixed address
    BEQ done        ; null terminator → exit
    JSR $FFD2       ; CHROUT
    INX
    CPX #$12        ; 18 characters max
    BNE for0
done:
    RTS
```

> **LOOP vs FOR:** LOOP counts down (N→1) — good for delays, fills, pixel loops. FOR counts up (0→N) — good for string/array access. Both can use X or Y.

PUSH / PULL

Like **saving variables before a GOSUB and restoring them after** — but uses the 6502 hardware stack. If a subroutine uses A, X, or Y, wrap it with PUSH and PULL so the calling code's registers are preserved.

PUSH

Pushes one or more registers onto the stack. The order is always A → X → Y (innermost first).

Field	Description
Registers	Any combination: `A`, `X`, `Y`, `AX`, `AY`, `XY`, `AXY`

Expert syntax:

```
.push AXY
```

Generated ASM (example: `AX`):

```
PHA
TXA
PHA
```

Size: 1 byte for A (`PHA`), 2 bytes for X or Y (transfer + push).

PULL

Restores registers from the stack in **reverse order** (Y → X → A).

Field	Description
Registers	Same as PUSH — must match the corresponding PUSH block

Expert syntax:

```
.pull AXY
```

Generated ASM (example: `AX`):

```
    PLA
    TAX
    PLA
```

> **Rule:** PUSH and PULL must always use the **same register set**. `PUSH AX` → `PULL AX` (internally restores in reverse: X first, then A).

END / RTS alias

Like **RTS with a friendlier macro name** — `.end` emits a single `RTS` byte and behaves as a short subroutine terminator in expert mode.

Expert syntax:

```
.end
```

Generated ASM:

```
    RTS
```

Size: 1 byte.

Use this when you want an end-of-subroutine marker that reads a little more like a macro than a raw instruction.

MACRO / ENDM / INVOKE

Like **a named GOSUB with parameters** — define a reusable chunk of code once (MACRO…ENDM), then call it anywhere with INVOKE. Pass different argument values each time instead of copy-pasting blocks.

MACRO (definition start)

Field	Description
Name	Identifier for the macro (e.g. `setColor`)
Params	Optional comma-separated parameter names (e.g. `color` or `color, count`)

Marks the start of a macro definition. Blocks between MACRO and ENDM are the macro's body — they **don't generate any code** where the definition sits. Use `{paramName}` as a placeholder for arguments.

Generated ASM:

```
; .MACRO setColor (color)
    ... (body blocks)
; .ENDM
```

Expert mode syntax:

```
.macro setColor color
    LDA {color}
    STA $D020
.endm
```

ENDM (definition end)

Closes the current macro definition. No fields.

INVOKE

Calls a defined macro at this position and substitutes the supplied argument values for `{paramName}` placeholders in the body.

Field	Description
Macro name	Select from the dropdown of defined macros
Arguments	Comma-separated argument values matching the macro's parameter list (e.g. `#$07`)

Generated ASM:

```
; .invoke setColor(#$07)
    LDA #$07
    STA $D020
```

Expert mode syntax:

```
; single argument:
.invoke setColor(#$07)

; multiple arguments:
.invoke drawPixel($10, $20)

; no arguments:
.invoke clearScreen

; text / string arguments (quoted):
.invoke printText("Hello, World!")

; .call alias (synonym for .invoke):
.call setColor(#$07)
```

The macro body is expanded inline with `{paramName}` replaced by the actual arguments. The space-separated form (`.invoke setColor #$07`) is also accepted.

Argument types:

- **Numeric**: `#$07`, `$10`, `255` — hex or decimal values
- **Text strings**: `"Hello, World!"` — quoted strings; commas inside quotes are treated as part of the text, not as argument separators
- **Mixed**: `#$07, "hello", $20` — any combination

> **Tip:** Define macros at the top (or bottom) of your program, then INVOKE them wherever needed. Macros can be invoked multiple times with different arguments.

REGION / ENDREGION

Purely visual grouping — **zero bytes**, zero effect on the assembled code. Like folding a section of a BASIC program into a named block so you can collapse it and focus on something else.

Field	Description
Region name	Free-text label for the section (e.g. `init`, `game_loop`, `sprite_setup`)

Expert syntax:

```
.region init
    ; blocks...
.endregion
```

Controls on the REGION block header (always visible):

- ▸ / ▾ **toggle** — collapses or expands the entire region. When collapsed, all blocks between REGION and ENDREGION are hidden.
- ↕ **Expand all** — un-collapses every individually collapsed block inside the region and expands the region itself if needed.
- ⊖ **Select in ASM** — highlights the entire region's code range in the ASM view (from `; ===[ name ]===` to `; ===[/name]===`) and scrolls to it. Switches to the ASM tab automatically if it is not currently visible.
- ⧉ **Copy region** — copies the REGION block, all child blocks, and the matching ENDREGION into a clipboard. A ✓ flash confirms the copy.
- ⎘ **Paste region** — inserts the copied region as a new region immediately after the current region's ENDREGION and scrolls to it. The button is dimmed until a region has been copied.

Generated ASM:

```
; region init
    SEI
    LDA #$00
    STA $D020
; endregion init
```

Size: 0 bytes for both REGION and ENDREGION.

Example workflow:

1. Add a `REGION` block, set region name to `init`.
2. Add your initialization instructions below it.
3. Add an `ENDREGION` block to close the section.
4. Click ▸ on the REGION to collapse the whole section into one line while working on other parts of the program.

> **Note:** Regions can be **nested** inside each other. Each ENDREGION closes the nearest open REGION. No effect on the assembled output.

DEFINE / IF / ELSE / ENDIF

Like **a switch the assembler reads** — `DEFINE DEBUG` turns on a symbol, then any `IF DEBUG` block is included and its `ELSE` branch is skipped. Remove the DEFINE block and the IF block disappears from the output. No need to delete code for release builds.

DEFINE

Field	Description
Symbol	One or more comma-separated identifiers to activate (e.g. `DEBUG` or `DEBUG, PAL`)

Expert syntax:

```
.define DEBUG, PAL
```

Generated ASM:

```
; .DEFINE DEBUG
; .DEFINE DEBUG, PAL
```

A `DEFINE` block can activate multiple symbols at once (comma-separated). Place DEFINE blocks at the top of your program. Removing the block deactivates all its symbols instantly.

IF

Field	Description
Condition	Identifier to test (must match a `DEFINE` symbol to be active)

Expert syntax:

```
.if DEBUG
```

Generated ASM:

```
; .IF DEBUG
```

Blocks between `IF` and `ENDIF` (or `ELSE`) are included or skipped based on whether the condition symbol has a matching `DEFINE` in the program. Skipped blocks appear as `; [IF skipped] …` comments and generate **zero bytes**.

ELSE

No fields. Marks the alternative branch — assembled when the `IF` condition is *not* active.

Expert syntax:

```
.else
```

Generated ASM:

```
; .ELSE
```

ENDIF

No fields. Closes the conditional block.

Expert syntax:

```
.endif
```

Generated ASM:

```
; .ENDIF
```

Size: 0 bytes for all four blocks. Only the content *between* them counts.

Example — debug border flash, release build skips it:

```
; .DEFINE DEBUG

; .IF DEBUG
    LDA #$02        ; red border
    STA $D020
; .ELSE
    LDA #$00        ; black (release)
    STA $D020
; .ENDIF
```

Example — multiple symbols in one DEFINE block:

```
; .DEFINE DEBUG, PAL

; .IF PAL
    LDA #$xx        ; PAL timing constant
; .ELSE
    LDA #$xx        ; NTSC timing constant
; .ENDIF
```

Nested `IF` blocks are supported. If an outer block is skipped, inner blocks are skipped too.

> **Note:** This is compile-time condition handling. For runtime compare/branch sugar, see **Runtime IF / ELSE / ENDIF** below.

CONST

Like **a named variable that never changes** — `SCREEN = $0400`. Use the name instead of typing raw addresses everywhere, making the code easier to read and change later.

Field	Description
Name	Identifier for the constant (e.g. `SCREEN`)
Value	Numeric value in the selected base (e.g. `0400` in HEX = address $0400), or a PC-relative expression (see below)
Format	HEX or DEC — controls how the value is entered and displayed

Expert syntax:

```
.const SCREEN = $0400
.const FRAMES_1S = 60
```

Generated ASM:

```
; .CONST SCREEN = $0400
```

The constant name appears in the **label picker** dropdown on instruction blocks — just click it to insert.

PC-relative expressions (`*+N` / `*-N`):

The value field also accepts `*+N` or `*-N`, where `*` is the compile address of the CONST block itself. This is used to create a named alias for a byte inside a nearby instruction — the classic self-modifying code pattern:

```
CONST op = *+1      ; op → address of the immediate operand of the next LDA
LDA #$00            ; $00 will be patched at runtime
...
STA op              ; overwrites the #$00 byte → LDA reads the new value next time
```

The CONST emits 0 bytes; the label resolves at compile time to `current_address + 1`.

Arithmetic expressions:

The value field accepts general arithmetic, including references to previously defined CONST names, hex/binary literals, and built-in math functions:

```
.const SCREEN      = $0400
.const SCREEN_END  = SCREEN + 40*25   ; 1000 bytes later
.const COLOR_RAM   = $D800
.const MID_X       = 160
.const SIN_TABLE   = round(sin(PI/8) * 127)   ; pre-computed sine value
```

Built-in functions: `sin()`, `cos()`, `round()`, `max(a,b)`, `min(a,b)`, `abs()`, constant `PI`

Operators: `+ - * /` Literals: `$FF` (hex), `%10110000` (binary) Low/high byte: `lo(expr)`, `hi(expr)`

Size: 0 bytes.

VAR

Like **CONST, but auto-allocated** — `VAR` reserves zero-page storage for a label without you having to type the address. Use it for counters, pointers, and short-lived state that belongs in ZP.

Field	Description
Name	Variable name / label
Size (optional)	Number of bytes to reserve. Omit for a single byte.

Expert syntax:

```
.var counter
.var timer, 2
```

```
.var lives
```

Practical example:

```
.region Vars
.var counter
.var timer, 2
.endregion

LDA #$00
STA counter
```

Generated ASM:

```
; .var counter
```

Size: 1 byte by default, or `N` bytes when size is specified.

The allocator walks a configurable zero-page cursor (`$02` to `$FE`) and assigns the next free slot. If the requested region overlaps an already-used label, the compiler emits a warning.

Runtime IF / ELSE / ENDIF

Like **a real branch template** — this version works at runtime, not compile time. It compares `A` , `X` , or `Y` against an immediate value and emits the correct `CMP` / `CPX` / `CPY` + branch sequence for you.

Field	Description
Register	`A` , `X` , or `Y`
Operator	`==` , `!=` , `<` , `<=` , `>` , `>=`
Value	Immediate value in HEX or DEC format

Expert syntax:

```
.if A == #$10
    LDA #$07
.else
```

```
        LDA #$0F
    .endif

```

Size: Depends on the chosen branches and compare form.

The comparison is unsigned by default. For `<=` and `>` the macro expands to the shortest equivalent branch chain for the selected register.

WHILE / ENDW

Like **a runtime loop with a test at the top** — the body runs while the condition stays true.

Field	Description
Register	`A`, `X`, or `Y`
Operator	`==`, `!=`, `<`, `<=`, `>`, `>=`
Value	Immediate value in HEX or DEC format

Expert syntax:

```

    .while A != #$00
        JSR getchar
    .endw

```

Size: Depends on the loop body and compare form.

Use `WHILE` when the loop may end before the first iteration finishes. It is the runtime counterpart of the count-based `LOOP / NEXT` helper.

REPEAT / UNTIL

Like **a runtime loop with a test at the bottom** — the body always runs at least once, then the condition decides whether to stop.

Field	Description
Register	`A`, `X`, or `Y`
Operator	`==`, `!=`, `<`, `<=`, `>`, `>=`
Value	Immediate value in HEX or DEC format

Expert syntax:

```
.repeat
    JSR getchar
.until A == #$00
```

Size: Depends on the loop body and compare form.

Use `REPEAT / UNTIL` when you want the body to execute at least once before the exit check.

MEMCPY / MEMSET

Like **small memory routines you reach for constantly** — `MEMCPY` copies a contiguous block, `MEMSET` fills a range with one byte.

Macro	Fields
`MEMCPY`	`src`, `dst`, `size`
`MEMSET`	`addr`, `value`, `size`

Expert syntax:

```
.memcpy src=$C000, dst=$D000, size=$0100
.memset addr=$0400, value=#$20, size=$03E8
```

Generated ASM: inline copy/fill loops, chosen to match the requested size.

Sizes up to 256 bytes use a short 8-bit loop. Larger sizes switch to a 16-bit counter automatically.

PRINT / PRINT_CHAR / PRINT_HEX / CLEAR_SCREEN / WAIT_KEY / DELAY / SET_BORDER / SET_BG

PRINT

Like **PETSCII output without the boilerplate** — prints a string through `CHROUT` with the same uppercase/lowercase handling as the PETSCII block. The lowercase checkbox is shared with the PETSCII encoder, so the text path stays consistent.

Expert syntax:

```
.print "HELLO"
```

```
.print "hello", lower
```

PRINT_CHAR

Prints one PETSCII byte by numeric code and sends it through `CHROUT`. The value can also be a named const or label that resolves to a byte at assemble time, in both Block mode and Expert mode.

Expert syntax:

```
.print_char 65
.print_char $41
.print_char color
```

PRINT_HEX

Prints an 8-bit value as hexadecimal text through the normal KERNAL output path.

Expert syntax:

```
.print_hex A
```

CLEAR_SCREEN

Shortcut for the standard C64 clear-screen control code.

Expert syntax:

```
.clear_screen
```

WAIT_KEY

Waits until a key is pressed, so you do not have to hand-roll the `GETIN` loop every time.

Expert syntax:

```
.wait_key
```

DELAY

Waits the requested number of frames through a shared helper routine. Use this for short pauses and timing gaps when a full custom loop would be overkill. The frame count can be a raw number or a named const, and `.wait` is just an alias of `.delay`.

Expert syntax:

```
.delay 29
.wait 29
.delay frames=FRAMES_1S
```

In Block mode the delay field uses a compact const picker when a symbolic value is available, so you do not have to type the name manually every time.

SET_BORDER / SET_BG

Convenience wrappers for the VIC-II color registers. The color value can be a raw number or a named const that resolves to 0–15. Both Block mode and Expert mode accept symbolic const names here.

Expert syntax:

```
.set_border 6
.set_bg 0
.set_border color
.set_bg color
```

Size: Each helper expands to a tiny register write sequence or a short KERNAL call.

In Block mode these fields also use the const picker, so the symbolic value stays visible instead of being replaced by a raw number.

IRQ_SETUP

Sets up a raster IRQ handler in one step. The macro writes the IRQ vector, enables raster IRQs, sets the line, disables the common CIA IRQ sources, and returns to normal execution with `CLI`.

Field	Description
Handler	IRQ routine label (e.g. `my_irq`)

Field	Description
Raster	Raster line in hex or decimal (e.g. `$FA`)

Expert syntax:

```
.irq_setup handler=my_irq, raster=$FA
```

Size: A small setup sequence; exact length depends on the selected raster line.

Use this when you want the common "SEI / install handler / enable IRQ / CLI" boilerplate without scattering it through the program.

RAND

Like **a tiny built-in PRNG** — returns an 8-bit pseudo-random value from a compact zero-page seed.

Field	Description
Seed	Optional zero-page seed byte or label (default `$FB`)

Expert syntax:

```
.rand
```

Size: A handful of bytes, depending on the chosen implementation path.

The generator is meant for gameplay, effect variation, and quick test data. It is deliberately small rather than cryptographically fancy.

SPRITE_INIT

Sets up a VIC-II sprite in one block — instead of writing ~6 POKE statements in BASIC, just fill in the fields. Sets the sprite's data pointer, turns it on, optionally enables multicolor mode, and sets its colour.

Field	Description
Sprite #	Sprite number 0–7
Colour	Colour index 0–15 (C64 palette)
Data page	Sprite data address / 64 (e.g. `$21` if data is at `$0840`)

Field	Description
Multicolor	Toggles the sprite's multicolor bit (`$D01C`)

Expert syntax:

```
.sprite_init 0, 7, $21
.sprite_init 0, 7, $21, multicolor
.sprite_init 0, 7, $21, mono
```

Generated ASM:

```
    LDA #$21
    STA $07F8       ; sprite pointer register ($07F8 + N)
    LDA $D015
    ORA #$01        ; set enable bit for sprite 0
    STA $D015
    LDA $D01C
    AND #$FE        ; clear multicolor bit for sprite 0
    STA $D01C
    LDA #$07
    STA $D027       ; sprite 0 colour register
```

Size: 26 bytes.

> **Sprite data page:** `data_address ÷ 64` . With the default BASIC SYS stub, `ALIGN 64` after `JMP main` places sprite data at `$0840` → page = `$21` .

SPRITE_POS

Like **`POKE 53248, x : POKE 53249, y`** in BASIC — sets a sprite's starting position. The coordinates are baked in at assemble time, and the fields accept consts in Block mode; for animation use `INC` / `DEC` on the sprite register directly.

Field	Description
Sprite #	Sprite number 0–7
X	Horizontal position 0–319
Y	Vertical position 0–255

Expert syntax:

```
.sprite_pos 0, 152, 100
```

Generated ASM (example: sprite 0, X=152, Y=100):

```
    LDA #$98        ; X low byte
    STA $D000       ; sprite 0 X register
    LDA $D010
    AND #$FE        ; clear X MSB for sprite 0 (X ≤ 255)
    STA $D010
    LDA #$64        ; Y = 100
    STA $D001       ; sprite 0 Y register
```

For X > 255 the macro sets the corresponding bit in `$D010` instead of clearing it.

Size: 18 bytes.

> **Note:** `SPRITE_POS` bakes the X/Y into the code (`LDA #$xx`). To animate a sprite at runtime use `INC $D000` / `DEC $D000` — see the `sprite-macro-demo` sample.

WAIT_RASTER

Waits for the VIC-II electron beam to reach a specific scan line — like syncing to a TV frame. Put this at the top of your game loop to prevent sprite tearing. No JSR, no label needed.

Field	Description
Raster line	Target raster line in hex (e.g. `FF` = line 255)

Expert syntax:

```
.wait_raster $FF
```

Generated ASM:

```
wait:
    LDA $D012       ; current raster line
```

```
    CMP #$FF        ; target line
    BNE wait        ; loop back (-7 bytes)
```

Size: 7 bytes (the `BNE` offset `$F9` = −7 always points back to the `LDA`).

> **Tip:** Place `WAIT_RASTER` at the top of your game loop to synchronise with the display and prevent sprite tearing.

JOYSTICK

Like reading `PEEK($DC00)` and then POKEing the sprite position — but in one block. Reads one CIA joystick port and adjusts a sprite's X/Y registers accordingly. Entirely inline, no JSR needed.

Field	Description
Port	`1` = port 1 (`$DC01`) or `2` = port 2 (`$DC00`)
Sprite #	Sprite number 0–7 (controls which X/Y register pair is updated)

Expert syntax:

```
.joystick 2, 0
```

Generated ASM (port 2, sprite 0):

```
    LDA $DC00       ; read CIA port 2
    LSR             ; bit 0 → carry (Up)
    BCS skip_up     ; carry set = NOT pressed
    DEC $D001       ; Y−1 (move up)
skip_up:
    LSR             ; bit 1 → carry (Down)
    BCS skip_down
    INC $D001       ; Y+1 (move down)
skip_down:
    LSR             ; bit 2 → carry (Left)
    BCS skip_left
    DEC $D000       ; X−1 (move left)
skip_left:
    LSR             ; bit 3 → carry (Right)
    BCS skip_right
    INC $D000       ; X+1 (move right)
skip_right:
```

Joystick bit map (active-LOW — bit = 0 means pressed):

Bit	Direction	CIA register
0	Up	$DC00 (port 2) / $DC01 (port 1)
1	Down	
2	Left	
3	Right	
4	Fire	(not handled by this macro)

Size: 27 bytes. The `BCS` offset is always `+3` (skips the following 3-byte `DEC` / `INC abs` instruction).

> **Typical usage:** Place inside a `gameloop` label with `WAIT_RASTER` first:
>
> ```
> gameloop:
>     WAIT_RASTER ($FF)
>     JOYSTICK (port=2, sprite=0)
>     JMP gameloop
> ```

MOUSE

Reads a Commodore 1351 proportional mouse and moves a sprite. Entirely **inline** — no JSR or label needed. The macro selects the CIA port, waits for the SID paddle inputs to settle, then decodes the delta movement using the standard 1351 driver pattern and applies it to the sprite registers.

Field	Description
Port	`1` = CIA `$DC00` bits `7:6` = `%01`; `2` = `%10`
Sprite #	Sprite number 0–7
ZP byte X	Zero-page address (hex) to hold the previous POTX sample (e.g. `FD`)
ZP byte Y	Zero-page address (hex) to hold the previous POTY sample (e.g. `FE`)

Generated ASM shape (port 1, sprite 0, ZP `$FD` / `$FE`):

```
    ; CIA port select + settle
    LDA $DC00
    AND #$3F
    ORA #$40
    STA $DC00
    LDX #$67
wait:
```

```
    DEX
    BNE wait

    ; X axis — standard 1351-style 7-bit delta decode
    LDA $D419
    TAY
    SEC
    SBC $FD
    AND #$7F
    LDX #$00
    CMP #$40
    BCS xneg
    LSR A
    BEQ xdone
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
    EOR #$01
    STA $D010
    JMP xdone
xneg:
    ORA #$C0
    CMP #$FF
    BEQ xdone
    SEC
    ROR
    DEX
    STY $FD
    CLC
    ADC $D000
    STA $D000
    TXA
    ADC #$00
    AND #$01
    BEQ xdone
    LDA $D010
    EOR #$01
    STA $D010
xdone:

    ; Y axis — same decode, then inverted before apply
    LDA $D41A
    TAX
    SEC
    SBC $FE
    AND #$7F
    CMP #$40
    BCS yneg
```

```
    LSR A
    BEQ ydone
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
    JMP ydone
yneg:
    ORA #$C0
    CMP #$FF
    BEQ ydone
    SEC
    ROR
    STX $FE
    EOR #$FF
    SEC
    ADC $D001
    STA $D001
ydone:
```

Size: 142 bytes.

Expert mode syntax:

```
.mouse port, spriteNum, zpX, zpY
; example:
.mouse 2, 0, FD, FE
```

Important: Before the first call, initialise the zero-page bytes with the current POTX/POTY values to avoid a jump on the first frame:

```
    ; port 1: LDA $DC00 : AND #$3F : ORA #$40 : STA $DC00
    ; port 2: LDA $DC00 : AND #$3F : ORA #$80 : STA $DC00
    LDA $D419 : LSR A : AND #$3F : STA $FD
    LDA $D41A : LSR A : AND #$3F : STA $FE
```

Tip: Poll the mouse once per frame — place `WAIT_RASTER` in the game loop before `MOUSE`.

SPRITE_COL

Like `PEEK($D01E)` in BASIC — checks the VIC-II hardware collision registers and tells you whether a sprite hit another sprite or the background. Entirely inline, no JSR needed.

Field	Description
Sprite #	Sprite number 0–7 (which sprite's bit to check)
Collision type	`Sprite-Sprite ($D01E)` — collision with another sprite; `Sprite-Background ($D01F)` — collision with background graphics

Expert syntax:

```
.sprite_col 0, sprite
.sprite_col 0, background
```

Generated ASM (sprite 0, sprite–sprite):

```
    LDA $D01E       ; read sprite-sprite collision register (clears it!)
    AND #$01        ; isolate bit 0 (sprite 0)
                    ; A ≠ 0 → collision occurred
```

Size: 5 bytes.

> **Important:** Reading `$D01E` / `$D01F` **clears the register**. Read it once per frame and act on the result immediately with `BEQ` / `BNE`.

Typical usage:

```
gameloop:
    WAIT_RASTER ($FF)
    JOYSTICK (port=2, sprite=0)
    SPRITE_COL (sprite=0, type=sprite-sprite)
    BEQ no_hit          ; A = 0 → no collision
    LDA #$02
    STA $D020           ; red border = hit!
    JMP gameloop
no_hit:
    LDA #$0E
    STA $D020           ; light blue border = clear
    JMP gameloop
```

> **See also:** `collision-demo` sample — green ball (sprite #0) vs. red cross (sprite #1).

LOADFILE

Like **`LOAD "file",8`** in BASIC — loads a file from a D64 disk at runtime using the KERNAL LOAD routine. Use this to load data, music, or extra code from disk while your program is running.

Field	Description
Filename	File name on the disk (max 16 chars, auto-uppercase; characters `,`, `"`, `/`, `\`, `:`, `*`, `?`, `<`, `>`, `|` are filtered)
Device	Device number 8–30 (default `8`)
Override address (optional)	Hex load address (e.g. `C000`). If set, the file is loaded to this address (`sec=0`, ignoring the PRG header). Leave empty to use the file's own 2-byte PRG header (`sec=1`).
Error label (optional)	If set, a `BCS` instruction is generated after JSR LOAD. If the KERNAL returns with carry set (error), execution jumps to this label.

Expert syntax:

```
.loadfile "DEMO-COLORS", 8
.loadfile "DEMO-COLORS", 8, $C000
.loadfile "DEMO-COLORS", 8, $C000, error_label
```

Generated code structure:

```
    JMP skip_filename      ; jump over the inline filename
    .byte "DEMO-COLORS"    ; filename bytes (PETSCII, 11 chars)
skip_filename:
    LDA #11                ; filename length
    LDX #<fname            ; pointer lo
    LDY #>fname            ; pointer hi
    JSR $FFBD              ; SETNAM
    LDA #1                 ; logical file #1
    LDX #8                 ; device 8
    LDY #0                 ; sec=0 (override addr) or #1 (file's own addr)
    JSR $FFBA              ; SETLFS
    LDA #0                 ; LOAD command (not VERIFY)
    JSR $FFD5              ; LOAD
    BCS fail               ; (only if error label set)
```

Size: `3 + filename_length + 9 (SETNAM) + 9 (SETLFS) + (4 if override) + 5 (LOAD) + (2 if error label)` bytes. Minimum 27 bytes.

> **Important:** The filename is stored inline in the machine code right after a `JMP skip_filename`. The file name on the disk must be uppercase PETSCII — which matches plain ASCII uppercase letters (`A` – `Z`). The macro enforces this automatically.

> **Always use an error label** for production programs — if the file isn't found, KERNAL sets the carry flag and execution falls through to whatever is next.

> **See also:** `loadfile-demo` sample — demonstrates loading `DEMO-COLORS.PRG` from a D64 with a BCS error branch and a visual error screen.

EXODECRUNCH

In-program **Exomizer decompression**. Use this macro right after a `LOADFILE` that loaded an Exomizer `mem`-mode compressed stream — EXODECRUNCH unpacks it backward into the address embedded in the stream.

Field	Description
Depacker address	Where the depacker code lives in memory (default `B000`). Must be a 16-bit hex address.

Expert syntax:

```
.exodecrunch
.exodecrunch depacker=$B000
```

Generated code (19 bytes):

```
    LDA $AE          ; KERNAL load-end lo (set by previous LOAD)
    STA $04          ; depacker src-end pointer lo
    LDA $AF          ; KERNAL load-end hi
    STA $05          ; depacker src-end pointer hi
    LDA #$36         ; BASIC ROM off ($A000-$BFFF becomes RAM)
    STA $01
    JSR depacker     ; depacker reads back-to-front via $04/$05
    LDA #$37         ; restore default mapping (BASIC + KERNAL + I/O)
    STA $01
```

How it works:

1. KERNAL `LOAD` ($FFD5) updates ZP `$AE/$AF` to point one past the last loaded byte. EXODECRUNCH copies this to ZP `$04/$05`, which is the official Exomizer backward source-end convention.

2. The depacker is typically placed at `$B000` (within the BASIC ROM mapped region). The macro toggles `$01 = $36` so the CPU sees RAM there during the JSR, then restores `$01 = $37` afterward.
3. The decompression target address is **encoded into the compressed stream itself** when you compress with `exomizer mem -l <load> file,<target>` — the depacker reads it from the stream's first bytes.

Depacker binary: the pre-built backward depacker is `samples/exo-decrunch.bin` (477 bytes, ORG $B000). It's a Kick Assembler wrap of the official `exodecrunch.asm` with `INC $D020` added to every read for a visible border-flash effect during decompression. Place it into your program with an `INCBIN` block at the depacker address.

Safety offset compensation: Exomizer's default mem mode applies a 2-byte safety offset — data lands 2 bytes earlier than the requested target. The Run via D64 dialog **automatically adds 2 to the Dst field** before calling exomizer, so the visible behaviour matches the address you typed.

> **See also:** the `exo-multicolor-demo` sample — full end-to-end example: LOADFILE a compressed multicolor bitmap to $C000, EXODECRUNCH unpacks it to $2000, then copy screen → $0400 and color → $D800, and switch VIC-II to multicolor bitmap mode.

> **Integration test:** `cargo test --test exomizer_integration` (in `src-tauri/`) verifies the full compression + decompression roundtrip on a 6502 emulator with the real depacker binary. Pass criteria: 10000 bytes byte-equal to the source `multi-color.bin`.

REU_CHECK

Detects whether a Commodore RAM Expansion Unit (REU) is plugged in — like checking `PEEK($D010)` to see if hardware is present. Tests by writing and reading back two patterns to REU register `$DF04`.

Field	Description
None	No operands

Generated code (34 bytes):

```
LDA #$55
STA $DF04
LDA $DF04
CMP #$55
BNE fail
LDA #$AA
STA $DF04
LDA $DF04
CMP #$AA
BNE fail
LDA #$00
CMP #$FF   ; Z=0 => REU present
BNE done
fail:
LDA #$FF
```

```
    CMP #$FF    ; Z=1 => no REU
    done:
```

> The macro normalizes the result so the following branch stays simple: `BNE` means REU present, `BEQ` means REU missing.

Expert syntax:

```
    .reu_check
```

Result in flags:

- **Z = 0** (result ≠ 0) → REU present → use `BNE`
- **Z = 1** (result = 0) → no REU → use `BEQ`

No configurable fields — the macro generates the same code every time.

Typical usage:

```
    REU_CHECK
    BEQ no_reu        ; skip if REU not present
    ; ... REU code here ...
    no_reu:
```

REU_STASH / REU_FETCH / REU_SWAP

DMA block transfer between C64 RAM and REU expansion memory — like a very fast POKE loop, but the CPU doesn't do any work (the REU chip copies the data while the CPU is halted). A `$1000`-byte transfer is effectively instant.

Macro	Direction	`$DF01` command
`REU_STASH`	C64 RAM → REU	`$90`
`REU_FETCH`	REU → C64 RAM	`$91`
`REU_SWAP`	C64 RAM ↔ REU	`$92`

Fields:

Field	Description	Example
C64 address	Source/dest in C64 RAM (hex)	`C000`

Field	Description	Example
REU address	Source/dest in REU (hex, 16-bit)	`0000`
REU bank	REU memory bank (0–7)	`0`
Length	Number of bytes to transfer (hex, 16-bit)	`1000`

Expert syntax:

```
.reu_stash $C000, $0000, 0, $1000
.reu_fetch $C000, $0000, 0, $1000
.reu_swap $C000, $0000, 0, $1000
```

Generated code (40 bytes):

```
LDA #c64lo    STA $DF02    ; C64 address LO
LDA #c64hi    STA $DF03    ; C64 address HI
LDA #reuLo    STA $DF04    ; REU address LO
LDA #reuHi    STA $DF05    ; REU address HI
LDA #bank     STA $DF06    ; REU bank
LDA #lenLo    STA $DF07    ; length LO
LDA #lenHi    STA $DF08    ; length HI
LDA #$00      STA $DF09    ; address control (fixed)
LDA #$00      STA $DF0A    ; interrupt mask (fixed)
LDA #cmd      STA $DF01    ; execute DMA ($90/$91/$92 = stash/fetch/swap, immediate)
```

> **Note:** Commands use `$90/$91/$92` (bit 4 set = immediate DMA mode). Writing to `$DF01` starts the transfer; the CPU resumes when it finishes.

TURBO_SET

Sets the **Ultimate-64 (U64) CPU speed** via register `$D031`. No effect on a real C64 or other emulators.

Fields:

Field	Description	Range
Speed	CPU speed index	0 = 1 MHz … 7 ≈ 10 MHz … 15 ≈ 48 MHz
Badline	Badline emulation	Enabled (C64 compatible) / Disabled (turbo)

The speed byte is calculated as: `(speedIndex & 0x0F) | (badline_disabled ? 0x80 : 0x00)`.

Generated code (5 bytes):

```
A9 xx   LDA #speed_byte
8D 31 D0   STA $D031
```

Expert mode syntax:

```
.turbo_set 7,0    ; speed=7 (~10 MHz), badline enabled
.turbo_set 15,1   ; speed=15 (~48 MHz), badline disabled
```

> **Note:** This macro affects U64 hardware only. On a real C64 or other emulators this writes to `$D031` which may affect the CIA or be ignored.

SUPERCPU_DETECT

Checks whether a **CMD SuperCPU** accelerator is installed — like `PEEK($D0B8)` to see if it returns something other than `$FF`.

Generated code (5 bytes):

```
AD B8 D0   LDA $D0B8
C9 FF      CMP #$FF
```

Result in flags:

- **Z = 0** → SuperCPU present → use `BNE`
- **Z = 1** → SuperCPU not found → use `BEQ`

No configurable fields.

Expert syntax:

```
.supercpu_detect
```

Typical usage:

```
SUPERCPU_DETECT
BEQ no_scpu       ; skip if SuperCPU not present
; ... SuperCPU turbo code here ...
no_scpu:
```

TURBO_ENABLE

Turns **CMD SuperCPU turbo mode** on or off. Call `SUPERCPU_DETECT` first and skip this if the SuperCPU isn't present.

Mode	Register	Effect
Enable	`$D07A`	Engage turbo (up to 20 MHz with SuperCPU)
Disable	`$D07B`	Return to 1 MHz compatibility mode

Generated code (5 bytes):

```
A9 00         LDA #$00
8D 7A D0      STA $D07A    ; (or $D07B for disable)
```

Expert mode syntax:

```
.turbo_enable on
.turbo_enable off
```

> **Note:** Call `SUPERCPU_DETECT` first and branch around this macro if the SuperCPU is not present.

MAP_COPY

Copies a tilemap from a source address to screen RAM (and optionally color RAM) using a series of `LDA abs,X` / `STA abs,X` loops. One 256-byte page is copied per loop iteration; a partial page at the end uses `CPX #rem / BNE` to stop. No JSR needed — all code is generated inline.

Field	Description
Source addr (screen)	Hex address where the map data lives after loading (e.g. `C000`)

Field	Description
Screen RAM dest	Where to copy screen codes (e.g. `0400`)
Size (bytes)	Total bytes to copy — typically `$03E8` = 1000 (40×25 chars)
Combined .bin	When checked, expects screen codes immediately followed by color data at `source + size`; copies color data to **Color RAM dest** in a second pass
Color RAM dest	Destination for color data — default `D800` (C64 color RAM)

Generated ASM (1000-byte map, screen only):

```
    LDX #$00
    LDA $C000,X   ; page 0
    STA $0400,X
    INX
    BNE *-9       ; loops until X wraps to 0 (256 iters)
    LDA $C100,X   ; page 1
    STA $0500,X
    INX
    BNE *-9
    LDA $C200,X   ; page 2
    STA $0600,X
    INX
    BNE *-9
    LDA $C300,X   ; remainder — 232 bytes
    STA $0700,X
    INX
    CPX #$E8
    BNE *-11
```

Size: `2 (LDX) + fullPages×9 + (rem > 0 ? 11 : 0)` bytes per section. Combined mode doubles it (screen section + identical color section).

Pairing with the Map Editor:

The Map Editor's **Files → Save map + color RAM (.bin)** exports a single binary where the first `size` bytes are screen codes and the next `size` bytes are color RAM values. Use MAP_COPY with **Combined .bin** checked and point **Source addr** at where this file is loaded (e.g. via INCBIN at `$C000`):

```
* = $C000
    INCBIN "map-color.bin" @ $C000   ; screen codes $C000–$C3E7, color $C3E8–$C7CF
* = $0801
    ; ...
    MAP_COPY src=$C000 dst=$0400 size=1000 combined color_dst=$D800
```

Expert mode syntax:

```
.map_copy $C000, $0400, 1000               ; screen only
.map_copy $C000, $0400, 1000, auto, $D800  ; combined (color at src+size)
.map_copy $C000, $0400, 1000, $C3E8, $D800 ; explicit color source address
```

MAP_COPY16X16

Copies a 16×16 character area from a compact 256-byte screen-code block plus a matching 256-byte Color RAM block. It is intended for Charset Canvas exports and small tile/picture chunks, where writing sixteen separate MAP_COPY rows would be noisy.

Default layout:

Data	Default address
16×16 screen codes	Source address (`src`)
16×16 color values	`src + 256`
Screen RAM destination	`$0400 + row×40 + col`
Color RAM destination	`$D800 + row×40 + col`

Expert mode syntax:

```
.map_copy16x16 $3000, 12, 4
.map_copy16x16 $3000, 12, 4, $0400, $3100, $D800
```

The short form copies screen bytes from `$3000`, color bytes from `$3100`, and places the 16×16 block at column 12, row 4. Valid top-left positions are `col = 0..24` and `row = 0..9`, so the full 16×16 area remains on the 40×25 C64 text screen.

Generated behavior:

- Generates sixteen inline row copies.
- Each row copies 16 screen bytes and 16 color bytes.
- No JSR is needed; the code is emitted directly at the macro position.
- Works with normal character mode and multicolor character mode; the Color RAM bytes carry each cell's character color / multicolor enable bit.

Typical pairing with Charset Canvas:

```
* = $0801
    SEI
    LDA #$00
    STA $D021
    LDA #$05
    STA $D022
    LDA #$0D
    STA $D023
    .map_copy16x16 $3000, 12, 4
loop:
    JMP loop

.incbin "16x16+ram+color.bin", $3000
.incbin "16x16+charset.bin", $2000
```

SPRITE_ANIM

Advances a sprite's animation frame each call and updates the VIC-II sprite data pointer. Store one byte per frame in a table (the sprite data page number = `data_address / 64`), point SPRITE_ANIM at it, and call it once per game frame — no JSR needed.

Field	Description
Sprite #	Sprite number 0–7
Frame list address	Hex address of the frame table — one byte per frame, each byte = sprite page (`data_addr / 64`)
Frame count	Total number of frames (1–255)
Frame ZP	Zero-page byte used as the frame counter (e.g. `FB`)

Generated ASM (sprite 0, 4 frames, ZP `$FB`, list at `$C100`):

```
    INC $FB         ; advance frame counter
    LDA $FB
    CMP #$04        ; frame count
    BCC *+6         ; if counter < count, skip reset
    LDA #$00
    STA $FB
    TAX             ; X = current frame index
    LDA $C100,X     ; load sprite data page for this frame
    STA $07F8       ; update sprite 0 pointer ($07F8 + sprite#)
```

Size: 19 bytes.

Typical usage:

```
frameTable:
    .byte $21, $22, $23, $24   ; 4 frames at $0840, $0880, $08C0, $0900

gameloop:
    WAIT_RASTER ($FF)
    SPRITE_ANIM (sprite=0, list=$C100, count=4, zp=$FB)
    JMP gameloop
```

Expert mode syntax:

```
.sprite_anim spriteNum, frameListAddr, frameCount, zpByte
; example:
.sprite_anim 0, C100, 4, FB
```

> **Tip:** Place the frame table as a RAWBYTES block at a fixed address. The counter ZP byte (`$FB`) must be initialized to `$00` before the first call. If your code uses `$FB` for something else, pick a free ZP location.

SCORE_BCD

Adds a fixed point value to a multi-byte BCD score stored in memory, then renders each digit to screen RAM as a screen-code character. Uses the 6502 decimal mode (`SED` / `CLD`) for carry-safe BCD arithmetic — no manual carry juggling needed.

Field	Description
Score address	Hex address of the BCD score bytes (e.g. `C200`). Low byte first.
Digits	Number of BCD bytes (each byte holds two digits: `$99` = "99"). `4` bytes = up to 99999999.
Add points	Decimal value to add per call (e.g. `100`).
Screen address	Where to write the digit screen codes (e.g. `0400`). One byte per digit (high nibble first).

Expert syntax:

```
.score_bcd $C200, 4, 100, $0400
```

Generated ASM (4 bytes, +100 pts, score at `$C200`, screen at `$0400`):

```
    SED             ; enable BCD / decimal mode
    CLC
    LDA $C200       ; byte 0 (digits 1–2)
    ADC #$00        ; low byte of 100 in BCD = $00
    STA $C200
    LDA $C201       ; byte 1 (digits 3–4)
    ADC #$01        ; mid byte of 100 in BCD = $01 (carry propagates)
    STA $C201
    LDA $C202
    ADC #$00
    STA $C202
    LDA $C203
    ADC #$00
    STA $C203
    CLD             ; back to binary mode

    ; render digits
    LDX #$00
digit_loop:
    LDA $C200,X
    PHA
    LSR : LSR : LSR : LSR  ; high nibble → low nibble
    ORA #$30        ; + '0' screen code
    STA $0406,X     ; right-justified: screen_addr + (digits*2 - 2) - X*2
    PLA
    AND #$0F        ; low nibble
    ORA #$30
    STA $0407,X
    INX
    CPX #$04
    BNE digit_loop
```

Size: `3 + digits×8` bytes (SED + CLC + CLD overhead + 8 bytes per BCD byte for ADC + display loop).

Expert mode syntax:

```
.score_bcd $C200, 4, 100, $0400
```

> **Tip:** Initialize the score bytes to `$00` at startup. The score address should be in zero page or absolute RAM — not ROM. The screen address should point to the leftmost digit cell; digits are written left-to-right (most-significant byte first).

> **BCD range:** `digits=4` bytes → 8 decimal digits → max score 99,999,999. Each byte encodes two BCD digits: `$00` – `$99`.

10. Debugger Integration

The app supports **RetroDebugger** as the external C64 debugger. It receives breakpoints, symbols, and autostart flags generated from the assembled program.

RetroDebugger

RetroDebugger is a cross-platform Commodore 64 debugger with breakpoint support, memory inspection, and label-aware disassembly.

Setup: Open **Settings** → **Configure RetroDebugger executable** and point it to the `RetroDebugger` binary.

Launch: Click **Debug (RetroDebugger)** in the toolbar. The app will:

1. Assemble the program to a `.prg` file in a temporary directory.
2. Write a **breakpoints file** (`breakpoints.txt`) — one `break $ADDR` per flagged block.
3. Write a **symbols file** (`symbols.txt`) in Vice/RetroDebugger label format (`al C:addr .name`). All LABEL and CONST blocks are included.
4. Also write C64Debugger-style sidecars next to the compiled PRG: `.dbg` , `.sym` , and `.vs` .
5. Launch RetroDebugger with:

```
RetroDebugger -prg <file.prg> -breakpoints <breakpoints.txt> -symbols <symbols.txt> [flags]
```

Breakpoint Blocks

Click the breakpoint icon (●) on any instruction block to toggle it as a breakpoint. Breakpointed blocks are highlighted in red. Their addresses are written to the breakpoints file on every debugger launch.

Debugger Flags (Options Tab)

Toggle	Flag	Effect
`-jmp` ON	`-jmp $ADDR`	Jump directly to the program start address after loading
`-unpause` ON	`-unpause`	Unpause the debugger immediately on load
`-wait` ON	`-wait <ms>`	Wait `<ms>` milliseconds before unpausing — 500 ms or 1000 ms

> **Tip:** For most programs, enable `-jmp` and `-unpause` for instant autostart. Use `-wait 500` or `-wait 1000` when your program sets up IRQs or SID music that needs time to initialize before the first raster.

11. Knowledge Base Links

Quick reference links available in the app under **Knowledge Base**:

Resource	URL
6502 Opcodes Reference	http://www.6502.org/tutorials/6502opcodes.html
C64 KERNAL Functions	https://sta.c64.org/cbm64krnfunc.html
C64 Memory Map	https://sta.c64.org/cbm64mem.html
C64 Color Codes	https://sta.c64.org/cbm64col.html
VIC-II Article	https://www.cebix.net/VIC-Article.txt
C64 Codebase	https://codebase.c64.org/
The Turbo Assembler	https://turbo.style64.org/
RetroDebugger	https://github.com/slajerek/RetroDebugger/

---

12. D64 Export & Run

Version 1.5.1 adds the ability to package your program (and additional data files) into a C64 D64 disk image and launch it in VICE — or export the disk image for use elsewhere.

Split Run button

The toolbar **Run** button has been replaced by a **split button**:

Part	Action
▶ **Run** (main)	Executes the currently selected run mode
▾ (arrow)	Opens the mode selector

Available run modes:

Mode	Description
Run as PRG	Assemble to a temporary `.prg` and launch VICE directly. Classic behaviour.
Run via D64	Assemble, build a `.d64` disk image (using c1541), add any configured extra files, then launch VICE from the disk. Use this whenever your program loads files at runtime (e.g. with the LOADFILE macro).
Run on hardware	Assemble to PRG and send it to a **1541 Ultimate / Ultimate 64** device over the local network. See Section 13.

The selected mode is saved between sessions.

Export to D64 dialog

Open via the **Save PRG** ▾ dropdown → **Export to D64**. The dialog lets you:

1. Set the **disk name** (max 16 chars) and **program name** — these are the names that appear in the C64 disk directory.

2. **Add extra files** — click **+** to pick any binary file (`.prg` , `.bin` , `.sid` , etc.). For each extra:
    - **Name** — how it appears in the D64 directory (max 16 chars, auto-uppercase).
    - **Addr** (load address, optional) — if provided, a 2-byte PRG header is prepended. Leave empty to write raw bytes with no header.
    - **Dst** (decompression target, only with EXO) — where the depacker should land the data on the C64. When EXO is enabled, the extra is compressed with `exomizer mem -l <Addr> file,<Dst>` before being written to the D64. The +2 safety-offset compensation is applied automatically.
    - **EXO** — checkbox that turns on backward `mem` -mode crunching for this entry. The on-disk size is typically 5-20% of the original.
3. Click **Export** to generate the `.d64` file using VICE's `c1541` tool.

Pairing with EXODECRUNCH: when you ship a file with EXO=on, the program reading it should LOAD it to the **Addr** address (sec=1, file's own PRG header), then call the **EXODECRUNCH** macro to unpack it backward into **Dst**. See the `exo-multicolor-demo` sample for the complete pattern.

D64 metadata in projects

The disk name, program name, and extra file list are saved inside the project JSON (under the `d64` key). When you reload the project or a sample that includes D64 metadata, the extras are restored automatically — no need to re-add them each time.

The **loadfile-demo** sample comes pre-configured with `DEMO-COLORS.PRG` as an extra file. Select it, open **Run via D64**, and click **Run** to see the full load flow in action.

> **Requirement:** D64 export and Run via D64 both require VICE (`c1541`) to be configured in Hardware Settings.

12b. CRT Export (Magic Desk 64K cartridge)

Menu → Build → Build CRT produces a Commodore 64 cartridge image (`.crt` , **cartridge type 19 — Magic Desk / Domark / HES Australia**) that runs on VICE, TheC64, real hardware via EasyFlash / Kung Fu Flash, and 1541 Ultimate II+ cartridge slots. It is available in both **block mode** and **Expert mode**, and — as of the current build — in **UltimateBasic mode** as well.

What ships in the cart

- **8 × 8 KB banks** at `$8000` , bank-switched via `$DE00` (Magic Desk convention: low 3 bits = bank, bit 7 = disable cart).
- **Bank 0** contains a 128-byte header + boot loader:
    - `$8000/$8002` cold + warm start vectors point at `$8009` .
    - `$8004–$8008` = the `CBM80` signature required by the KERNAL reset code.
    - `$8009–$807F` = the loader: SEI / stack init / `JSR $FDA3` (IOINIT) / `JSR $FD50` (RAMTAS) / `JSR $FD15` (RESTOR) / `JSR $FF5B` (CINT), then a byte-copy loop that streams the payload from cart ROM into RAM and switches banks when `$FC` reaches `$A0` . At the end it copies a tiny **exit stub** to `$0100` , disables the cart with `LDA #$80 : STA $DE00` , and `JMP` s to the entry point.
- **Payload** starts at `$8080` in bank 0 and spills into banks 1–7 as needed. Maximum payload = `8 * 8192 − 128 = 65 408 bytes` .

Load address and entry point

Build CRT never uses Exomizer (the depacker cannot run from cart ROM). It compiles the current tab with the standard autostart pipeline and takes the load address from the PRG header and the entry point from the SYS target:

- **Block / Expert mode with BASIC SYS stub on:** load = `$0801`, entry = the SYS target (typically `$080D` or the user's origin).
- **Block / Expert mode with BASIC SYS stub off:** load = user origin (with the classic `$0801 → $C000` fallback), entry = load address.
- **UltimateBasic mode:** load and entry both come from the UB compiler map (`build.map.loadAddress`). The UB autostart stub inside the payload is then executed exactly as it would be after `LOAD "...",8,1 : RUN` from disk.

The origin address you see in the ASM output is preserved; the loader simply copies the flat memory image from the PRG into RAM and jumps to the entry point once cart ROM is unmapped.

Size limit

Because the payload is stored linearly and `assembleProgramToPrg()` returns a flat `minAddr..maxAddr` buffer with zero-filled gaps, a program with widely spaced ORG segments (e.g. `$0801` + `$C000` + `$E000`) counts every byte in between toward the 65 408-byte budget. If you exceed the limit the build aborts with a `saveCrtTooLarge` error — either compress the memory layout or split the data.

> ⚠️ **Prominent caveat — read before shipping a CRT**
> The loader calls the KERNAL `RESTOR` **($FD15)** as part of the standard reset sequence. This intentionally rewrites the standard I/O vectors at `$0314/$0315`, `$0316/$0317`, `$0318/$0319`, `$0328/$0329` and friends back to their ROM defaults. Consequences:
>
> - **Any IRQ / NMI / BRK hooks set before the CRT boots are wiped.** Your program must install them itself after entry — exactly like a fresh `LOAD "",8,1 : RUN` from tape/disk.
> - **UltimateBasic programs** that rely on non-default KERNAL vectors being live at entry may need an explicit `SYS` or init call in the autostart stub. The standard UB autostart works out of the box; extension libraries that hook vectors *before* `RUN` do not.
> - **CIA1 / CIA2** are re-initialised by `IOINIT`. Custom timer setups (raster IRQs, music player CIA-A) must be reprogrammed after entry.
> - The cart is disabled from an 8-byte stub in **RAM at `$0100`** so that the `STA $DE00` cannot be interrupted by a bad ROM fetch. Do not rely on `$0100–$0107` containing the stack top image on entry — the first RAM push overwrites the stub.
>
> If a CRT runs in VICE but fails on real hardware, the first thing to check is whether the program assumes a specific KERNAL vector or CIA timer state at entry. Install the state explicitly in your init routine and it will behave the same on both.

Compatibility

Platform	Status
VICE (`x64sc`, `x64`)	Works via **File → Attach cartridge image**.
TheC64 / TheC64 Mini	Works via the built-in cartridge loader.
Kung Fu Flash	Works — native Magic Desk mode.
EasyFlash cartridge	Works when programmed as Magic Desk.
1541 Ultimate II+ / Ultimate 64	Works via **Cartridge → Load cart image**.

Platform	Status
Chameleon / Turbo Chameleon	Works.

---

13. Hardware Settings

Open via **Settings** → **Hardware Settings...** in the toolbar menu. All external hardware paths and network configuration live here.

VICE Emulator

Setting	Description
Select VICE	Browse to the `x64sc` (or `x64`) VICE executable
Status	Shows whether the executable path is valid and accessible

VICE is required for **Run as PRG**, **Run via D64**, and **Export to D64**.

Exomizer

Setting	Description
Select Exomizer	Browse to the `exomizer` executable
Border flash during decompression	When enabled, SFX-compressed PRGs use exomizer's built-in `-x1` fast border-flash effect; when disabled, `-n` is passed for silent decompression
Status	Shows whether the executable path is valid and accessible

Workflow:

1. Install the Exomizer binary:
   - **Windows:** download the pre-built `win32/exomizer.exe` from https://bitbucket.org/magli143/exomizer/wiki/Home or https://csdb.dk/release/?id=244342.
   - **macOS:** `brew install exomizer` (installs Magnus Lind's official 3.1.2 build).
2. Configure the path in **Hardware Settings** → **Exomizer section**.
3. Enable the **Run with Exomizer** checkbox in the **Settings menu**.
4. All **Run** actions (PRG, D64, hardware) and **Build** actions (Build PRG, Build D64) will now crunch the assembled program through `exomizer sfx sys` before launching or saving.

Exomizer works the same way on Windows and macOS — the CLI is invoked from the Tauri backend; nothing about the integration is platform-specific.

Two compression modes are used internally:

Mode	Used by	Calling convention
`sfx sys`	Build/Run with Exomizer toggle (main PRG path)	Self-extracting PRG with a built-in decruncher; the Border-flash setting controls `-x1` vs `-n`

Mode	Used by	Calling convention
`mem` (backward)	Run via D64 → per-file **EXO** checkbox	Compresses each extra file to a `mem`-mode stream; the program decompresses it at runtime via the **EXODECRUNCH** macro and a pre-built depacker (`samples/exodecrunch.bin`)

> **Tip:** If the Exomizer path is not configured but the checkbox is enabled, a clear error toast is shown instead of launching. Disable the checkbox to run without compression.

> **Integration test:** `cargo test --test exomizer_integration` (in `src-tauri/`) verifies the full mem-mode compression + decompression roundtrip on a 6502 emulator.

Retro Debugger

Setting	Description
Select RetroDebugger	Browse to the `RetroDebugger` binary
Status	Shows whether the path is valid

See Section 9 for full debugger documentation.

C64 Ultimate / 1541 Ultimate

Run assembled PRGs directly on real hardware over the local network using the Ultimate REST API.

Setting	Description
Host (IP)	IP address of the device (e.g. `192.168.1.100`)
Password	Optional — if the device requires authentication
Test connection	Sends a test request to `/v3/runners/info` ; shows OK or error

Workflow:

1. Connect the 1541 Ultimate / Ultimate 64 to your local network.
2. Enter its IP address (and password if set) in Hardware Settings.
3. Select **Run on hardware** from the split run menu.
4. Click ▶ **Run** — the PRG is compiled and sent to the device via HTTP POST to `/v3/runners/prg` . The device loads and runs it on the C64 immediately.

> **Tip:** No USB cable or driver needed — the REST API is built into the Ultimate firmware. Your computer and the device must be on the same local network.

---

14. Visual Editors (Toolkit)

The toolbar **Toolkit** menu groups the visual data editors that all share a common Files menu (`Files ▾`) for Load BIN / Save BIN / Export to blocks / Save to D64. Each editor produces raw `.bin` data that can be placed in a program with `INCBIN` , or directly added to a D64 disk via the **Save to D64** entry.

Visual-editor dialogs can be dragged by their headers across the full Visual Assembler workspace. A dialog with no saved position opens centered; after it is moved, its last position is stored in UI settings and restored the next time it opens.

Hi-Res / Multicolor Editor

Pixel-level bitmap editor with both 320×200 hi-res and 160×200 multicolor modes. Open via Toolkit → Hi-Res Editor.

Feature	Description
Mode toggle	**Multicolor** checkbox switches between hi-res (mono per cell) and multicolor (4 colors per cell).
Tools	Pencil, eraser, line, rectangle, filled rectangle, oval, filled oval, flood fill.
Spray tool	Airbrush-style painter that scatters pixels around the cursor while drawing.
Spray intensity	Dropdown next to the spray tool controls how dense each spray stroke is.
Color palette	Foreground (ink) + paper (background) pickers. Multicolor mode tracks 3 per-cell extras automatically.
Undo / Redo	Per-stroke history, ctrl-Z / ctrl-Y.
Grid + Raster	Optional 8×8 grid and raster row overlay for cell alignment.
Import image	PNG/JPEG/GIF dropped into the canvas auto-quantizes to the 16-color C64 palette.
Export blocks	Appends BYTE/RAWBYTES blocks to the program with the encoded bitmap, screen, and color data.
Export `.bin`	Saves the native multicolor format (10000 bytes: 8000 bitmap + 1000 screen + 1000 color) ready for LOADFILE to $2000.

Sprite Editor

24×21 pixel sprite editor with multi-frame animation. Open via Toolkit → Sprite Editor.

Feature	Description
Frames	Add / remove / reorder frames; frame strip shown at the bottom.
Mode	Mono / multicolor toggle.
Tools	Pencil, fill, line, rectangle, circle — plus flip horizontal, flip vertical, shift left/right/up/down (with optional wrap). Shape tools show a live preview while dragging; release to commit.
Undo / redo	Full undo/redo stack per frame. Ctrl/Cmd+Z / Ctrl/Cmd+Y or the toolbar buttons.
Image import	Import a PNG or JPEG via Files → Import image. The importer maps each pixel to the nearest C64 palette colour and writes it into the current frame.
Animation preview	Play / Stop with configurable speed.
Export blocks	Inserts a RAWBYTES block at a 64-byte-aligned address for every frame, plus a sprite pointer setup.
Save `.bin`	Writes 64 bytes per frame (raw sprite data without padding).

C64 Character ROM Browser ("Char map")

Read-only viewer of the C64 character ROM (the built-in PETSCII font). Open via the **C64 chargen** entry in the Toolkit menu. Useful for finding the screen-code of a glyph before writing it with `RAWBYTES` or `TEXT`.

Feature	Description
Two character sets	Tab 1: **Set 1 — Upper/Graphics** (default mode after power-on). Tab 2: **Set 2 — Lower/Upper** (after `$0E` switch).
Glyph grid	16×16 grid of all 256 characters. Click a glyph to see its detail panel: zoomed 8×8 pixel view, screen code (decimal + hex), PETSCII codes (both default and shifted), and the raw 8-byte bitmap.
Detail panel	Shows the selected glyph's screen code, PETSCII codes, and the eight raw bytes — ready to paste into a `RAWBYTES` or BYTE block.
Read-only	No editing here — use the Character Editor (below) to modify glyphs.

Character Editor (Charset)

256-character 8×8 charset editor. Open via Toolkit → Character Editor.

Feature	Description
Load ROM	Imports the C64 ROM charset directly from VICE's `chargen` (no file picker).
Load `.bin`	Imports an external 2048-byte charset binary.
Per-char preview	16-wide grid of all 256 glyphs with the current cell highlighted.
Pixel editor	8×8 single-character editor with toggle/invert/clear tools.
Per-character color	Stores a default Color RAM value for each glyph. In multicolor character mode this also preserves the per-cell multicolor enable bit and the character's own lower-3-bit color.
Metadata round-trip	When loading compatible data from Charset Canvas / Map workflows, per-character color metadata is preserved so edits can continue without losing color intent.
Export blocks	Appends RAWBYTES at $0800 (block 2) or $3800 (block 7) with the encoded charset.

Charset Canvas Editor

Full-canvas charset painter for building screens from a complete 256-character charset. Open via Toolkit → Charset Canvas.

The canvas is 16×16 characters. In mono mode that gives a 128×128 pixel working area; in multicolor character mode it gives a 64×128 wide-pixel working area using the C64's real character multicolor rules.

Feature	Description
Mono / multicolor	Mono mode stores 1-bit 8×8 glyphs. Multicolor mode stores two-bit horizontal pixel pairs and marks used cells with Color RAM bit 3.
C64 color model	Background uses `$D021`; shared multicolor 1 uses `$D022`; shared multicolor 2 uses `$D023`; each character's own color comes from Color RAM bits 0–2.
Drawing tools	Pencil, eraser, line, rectangle, oval, and flood fill work across character boundaries.
Spray tool	Airbrush-style drawing that scatters pixels across neighboring cells/characters.
Spray intensity	Dropdown next to the spray tool sets the stroke density.
Grid toggle	The Grid checkbox shows or hides the 16×16 character grid.

Feature	Description
Save charset `.bin`	Saves the 2048-byte character bitmap data.
Save 16×16 map + Color RAM `.bin`	Saves 256 screen codes followed by 256 Color RAM values. Use this with `MAP_COPY16X16`.
Loading	Can load charset-canvas saves, plain charset data, and compatible Character Editor charset data including stored per-character colors when present.

Important C64 limitation: in multicolor character mode the two shared colors are global for the whole screen (`$D022` / `$D023`). Only the character's own color is per cell, and it is limited to colors 0–7 because Color RAM bit 3 selects multicolor mode.

Map Editor (Multilayer Tilemaps)

Layered tilemap editor for static scenery, sprite spawn maps, collision data, and similar. Open via Toolkit → Map Editor.

Feature	Description
Layers	Multiple named layers, each with its own tileset and opacity.
Brushes	Single-tile, fill, line, rectangle, and circle modes. Shape tools show a live preview while dragging; release to commit.
Undo / redo	Full undo/redo stack per layer. Ctrl/Cmd+Z / Ctrl/Cmd+Y or the toolbar buttons.
Clear menu	Per-layer or whole-map clear with confirmation.
Image import	Drop a PNG of a tilemap; the editor auto-slices into tiles.
Copy / paste	Copy a selected tile region, then paste normally or use transparent paste to keep empty tiles transparent.
Multicolor-aware tile coloring	With compatible charset metadata, painting uses the tile's stored Color RAM defaults (including multicolor-related encoding) instead of a generic flat color.
Custom charset colors	When a charset carries `charColors` metadata, the Map Editor uses the selected tile's default Color RAM value while painting.
Export blocks	Emits RAWBYTES blocks for tileset graphics + map data.
Save Screen RAM (.bin)…	Saves only the screen codes for the current map layer (40×25 = 1000 bytes).
Save Screen RAM + Color RAM (.bin)…	Saves screen codes concatenated with Color RAM values as a single 2000-byte file (`screen[0..999]` followed by `color[0..999]`). Use this with the **MAP_COPY** macro (Combined .bin mode) to restore both screen and color in one operation at runtime.

SID Editor (3-Voice Tracker)

Multi-instrument 3-voice tracker with a Web Audio preview engine. Open via Toolkit → SID Editor.

Per-instrument controls:

- Waveform checkboxes (TRI / SAW / PUL / NOI) — multiple waveforms can be ORed together.
- ADSR (attack / decay / sustain / release) shown as a drag-graph above the four sliders.
- Pulse width slider (0-4095) with optional ring/sync flags.
- Filter routing checkbox per voice; global filter cutoff / resonance / volume / mode (LP/BP/HP).

Tracker grid:

- 3 voices × up to 7 patterns × 32 rows = 7 × 32 = 224 rows max (8-bit row counter constrains it).
- Per-row: note + instrument index. Empty rows hold the previous note.
- Select one cell normally, or hold **Shift** while clicking or using the arrow keys to extend a rectangular selection across rows and any of the three voices. Right-clicking inside the selected area keeps the range intact.
- Copy, Cut, Paste, and Clear are available from the icon toolbar and the icon-based context menu. `Ctrl/Cmd+C` and `Ctrl/Cmd+V` operate on the same rectangular selection.
- Harmony helper: choose a root note, chord type, and octave, preview the chord with the current instrument, then insert the voicing directly into the tracker. Available types include Major, Minor, Diminished, Augmented, Sus2, Sus4, Dominant 7, Major 7, Minor 7, 6, Minor 6, 9, b9, #9, Dim7, and 7sus4.
- Arpeggio helper: preview or insert 4-, 8-, or 16-step note runs from the selected chord in up, down, or up/down directions.
- **Preview row** auditions the selected row across all three voices without starting pattern playback.
- Cell-range paste now starts at the selected range start cell and stops cleanly at voice and row boundaries instead of wrapping into the next column or row.
- Speed slider sets the IRQ tick divisor (frames between rows).

Playback and virtual keyboard:

- The Play toolbar button changes to Pause during playback and to Resume while paused; Stop ends playback and resets the state.
- The keyboard toolbar button opens a non-modal piano that remains usable while the SID editor is active. Drag its header to place it anywhere over the main application.
- Enable **Insert into tracker** to write each played note at the current tracker cursor and advance to the next row. Disable it to audition notes without editing.
- Chord and arpeggio previews illuminate the corresponding piano keys when the keyboard is open.

Files menu exports:

Export	What it does
`Save .bin…`	Writes the editor's native serialized format (instruments + patterns + sequence).
`Export blocks (data only)`	Appends instrument table + pattern blocks to the program at `* = $C000`.
`Export blocks + miniplayer`	Adds the full player (sid_init / sid_irq / sid_play_row / sid_set_voice) plus PAL frequency tables. After export, place a `JSR sid_init` in your main code where the music should start.
`Export asm (clipboard)`	Copies the full assembly source to the clipboard.

Player ZP usage: `$FB` (tick counter), `$FC` (row index), `$FD` (set_voice temp). These conflict if your main code uses them — relocate via Expert mode if needed.

Known limits:

- Single linear pattern list (no per-voice sequence table yet).
- 8-bit row counter limits to 7 patterns × 32 rows.
- C64 `$D418` global volume is shared across voices — the per-instrument volume slider is informational; sustain level (`S` of ADSR) is the effective per-voice volume.
- Web Audio preview is approximate: PWM modulation, ring/sync, and the SID filter character differ from the real chip.

Curve Editor

Generates ready-to-use `.byte` lookup tables from mathematical curves — sine, easings, triangle/sawtooth/square, and bounce. Ideal for sprite movement, raster effects, colour cycling, or any animation driven by a precomputed table. Open via the **Curve Editor** icon in the top toolbar (next to the SID Editor button).

Curves:
Sine, Cosine, Linear, Ease In/Out/InOut (Quad and Cubic), Ease In/Out (Circ), Triangle, Sawtooth, Square, and Ease In/Out/InOut Bounce.

Controls:

Control	Purpose
Start / End value	Output range. 0..255 in 8-bit mode, 0..320 in 16-bit mode.
Number of values	Table length, 4–512 entries.
Cycles	How many oscillations across the table (sine/cosine/triangle/sawtooth/square only). Accepts fractions (e.g. `3.625`).
Phase	Phase offset in degrees (sine/cosine only).
Combine second curve	Blend a second curve with **Mix / Add / Multiply / Min / Max / Subtract**, its own cycles/phase, and a mix amount. Both source curves are drawn as dashed guides on the graph.
Label	Table label (auto-suggested from the curve name).
Number format	`$XX` hex or decimal.
Values per line	8 / 16 / 32 bytes per `.byte` line.

Output modes:

Mode	Emits
8-bit	A single `.byte` table (values 0..255). Read with `LDX #index / LDA table,X`. Optionally emits a **sprite-Y reader routine** (`<label>_set_y`) — `LDA <label>,X` / `STA $D001+2N` — for a selectable sprite number 0-7.
16-bit	Two parallel byte tables — `<label>_lo` (low 8 bits) and `<label>_hi` (9th bit, 0/1) — indexed by the **same** X (2 bytes per entry). Needed for sprite X across the full screen (0..320 > one byte). Optionally emits a **sprite-X reader routine** (`<label>_set_x`) that writes the low byte to `$D000+2N` and sets/clears the sprite's MSB in `$D010`, for a selectable sprite number 0-7.

Every Copy / Insert output starts with a header comment documenting the curve, actual min/max range, entry count, and exact usage (which register each table feeds).

Preview:

- **Graph** — the curve plotted with value 0 at the **top** and max at the **bottom**, matching the C64 sprite-Y / raster convention (so what you see is what the table drives on hardware). A meta line under the graph shows the byte count, the actual Min / Max of the generated values, and the curve name(s).
- **Bouncing ball** — animates a marker through the table at the **Tempo** (5–240 values/sec). At tempo 50 this equals one value per frame on PAL (50 Hz), i.e. one `.wait_raster` step per index. Play/Pause and Restart buttons, a

fading trail of the last ~24 positions, and a live `Index · Value` readout.

Copy / Insert: the toolbar's two icons — **Copy** puts the table on the clipboard; **Insert into editor** appends the table (and reader, if enabled) as blocks to the current program. Re-inserting **replaces** the previous Curve Editor insert instead of stacking duplicates (works in Block and Expert mode).

Files menu:

Action	What it does
Save curve (.bin)…	Saves the raw table bytes exactly as the C64 would read them via `INCBIN`. 16-bit: N lo bytes followed by N hi bytes.
Load curve (.bin)…	Loads raw table bytes back into the editor, interpreted per the current bit depth (16-bit: first half lo, second half hi). The loaded table is shown as-is until any curve control regenerates a fresh curve.
Export demo to blocks	Appends a complete, runnable sprite demo: sprite init, raster-synced main loop, the embedded table, and ball sprite data. X sweeps 0..320 in 8.8 fixed point with the `$D010` MSB while the table drives sprite Y — exactly matching the editor preview. Re-export replaces the previous Curve Editor insert.

Matching the preview on the C64: the preview reads the table **linearly, looping 0 → N-1 → 0, one value per frame**. To reproduce it exactly, drive the table the same way (increment the index once per frame, wrap at the table length). A ping-pong or partial-range playback will move differently even though the byte values are identical. See `samples/curve-new-demo.asm` for a working 16-bit sprite-X example.
